Editorial Management And Automation Platform (EMAP)



By

Sufyan Humam Mushtaq

B22F0158SE034

Abuzar Khan

B22F1053SE023

Jawad Ul Islam

B22F1235SE109

Final Year Project Report

School of Computing Sciences
Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology

Fall 2022



Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Editorial Management and Automation Platform (EMAP)

A Final Year Project Report Presented to

Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

In partial fulfillment

of the requirement for the degree of

Software Engineering

By

Sufyan Humam Mushtaq

B22F0158SE034

Abuzar Khan

B22F1053SE023

Jawad Ul Islam

B22F1235SE109

Fall 2022

Editorial Management and Automation Platform (EMAP)

A Final Year Project Report submitted to the School of Computing Sciences as partial fulfillment of the requirements for the award of the degree of BS Software Engineering.

Sufyan Humam Mushtaq	B22F0158SE034
Abuzar Khan	B22F1053SE023
Jawad Ul Islam	B22F1235SE109

Academic Supervisor

DR. Rashid Naseem
Associate Professor, School of Computing Sciences

Pak-Austria Pak-Austria Fachhochschule: Institute of Applied

Sciences and Technology

Industry Supervisor

DR. Rashid Naseem
Associate Professor, School of Computing Sciences

Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology

Haripur

Final Approval

This final year project titles

Editorial Management and Automation Platform (EMAP)

By

Sufyan Humam Mushtaq	B22F0158SE034
Abuzar Khan	B22F1053SE023
Jawad Ul Islam	B22F1235SE109

under the supervision of their project supervisor and approved by the project evaluation committee, has been accepted by the Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology, Pakistan, in partial fulfillment of the requirements for the degree of BS Software Engineering

Academic Supervisor:

_____

Dr. Rashid Naseem,
Associate Professor,
School of Computing Sciences

Industry Supervisor (if applicable):

_____

Dr. Rashid Naseem,
Associate Professor,
School of Computing Sciences,
PAF-IAST, Haripur

HoD / Chairman:

_____

Dr. Rashid Naseem,
School of Computing Sciences,
PAF-IAST, Haripur

Declaration

We, Sufyan Humam Mushtaq, B22F0158SE034, Abuzar Khan, B22F1053SE023, and Jawad Ul Islam, B22F1235SE109, hereby declare that we have produced the work presented in this final year project report during the scheduled period of study. We also declare that we have not taken any material from any source except where properly referred, and that the amount of plagiarism is within an acceptable range. It is further declared that we have developed this project and the accompanying report entirely on the basis of our personal efforts made under the sincere guidance of our supervisor. No portion of the work presented in this report has been submitted in support of any other degree or qualification of this or any other University or Institute of learning. If found otherwise, we shall stand responsible..

Date: 22 June 2026

Signature:_____
Sufyan Humam Muhstaq
B22F0158SE034

Signature:_____
Abuzar Khan
B22F1053SE023

Signature:_____
Jawad Ul Islam
B22F1235SE109

Certificate

It is certified that **Sufyan Humam Mushtaq (B22F0158SE034), Abuzar Khan (B22F1053SE023), and Jawad Ul Islam (B22F1235SE109)** have carried out all the work related to this project under my supervision at the Department of **School of Computing Sciences**, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology, and the work fulfills the requirement for the award of **BS Software Engineering**.

Date: 29 April 2026

Supervisor:

Dr. Rashid Naseem
Associate Professor, School of Computing Sciences

Head of Department:

Dr. Rashid Naseem
School of Computing Sciences

DEDICATION

We dedicate this project to our beloved parents for their endless prayers, sacrifices, and unwavering support throughout our academic journey. We also dedicate this work to our respected teachers and supervisor, whose guidance, encouragement, and knowledge made this achievement possible. Finally, this project is dedicated to all students and researchers striving to improve academic systems through innovation and technology.

ACKNOWLEDGEMENTS

All praise is due to Almighty Allah, the Most Gracious and the Most Merciful, for granting us the strength and perseverance to complete this Final Year Project successfully. We would like to express our sincere gratitude to our respected supervisor, Dr. Rashid Naseem, for his continuous guidance, valuable suggestions, and unwavering support throughout the completion of this project. His mentorship and expertise greatly contributed to the successful development of EMAP. We are also thankful to the faculty of the School of Computing Sciences, Pak-Austria Fachhochschule: Institute of Applied Sciences and Technology, for providing us with the academic knowledge and technical environment necessary for this achievement. Finally, we deeply appreciate our parents, family members, and friends for their encouragement, patience, and moral support throughout this journey.

ABSTRACT

Editorial Management and Automation Platform (EMAP)

The Editorial Management and Automation Platform (EMAP) is a comprehensive web-based solution designed to digitize and automate the management of Final Year Projects (FYP), thesis submissions, and research publication workflows within academic institutions. Traditional academic management systems often rely on manual procedures, fragmented communication, and inefficient record-keeping, leading to delays, reduced transparency, and administrative challenges. EMAP addresses these issues by providing a centralized platform that streamlines submission, evaluation, approval, and reporting processes. The system offers dedicated role-based dashboards for students, supervisors, reviewers, and administrators, ensuring secure and efficient workflow management. Key features include FYP proposal submission, thesis management, research paper handling, automated notifications, reviewer assignment, plagiarism integration support, analytics dashboards, and centralized archival of academic records. The platform is developed using ASP.NET Core MVC with SQL Server for robust backend management and Bootstrap-based responsive interfaces for usability across devices. EMAP enhances institutional productivity by improving communication, reducing paperwork, and ensuring transparency through role-based access control and audit trails. The platform is scalable and adaptable for universities, research organizations, and government educational bodies such as HED and DOST. This project demonstrates how automation and digital transformation can significantly improve academic governance, decision-making, and scholarly workflow efficiency while laying a strong foundation for future integration with AI-based reviewer recommendation, blockchain auditability, and advanced analytics.

Table of Contents

Chapter 1: Introduction 17

1.1 Problem Statement 17

1.2 Business Opportunity 17

1.3 Objectives 17

1.4 Project Scope 18

1.5 Constraints 18

1.6 Feasibility 18

1.6.1 Technical Feasibility 18

1.6.2 Resource Feasibility 19

1.6.3 Time Feasibility 19

1.6.4 Market Feasibility 19

1.7 Stakeholders Description 19

1.7.1 Stakeholders Summary 19

1.7.2 Key High-Level Goals and Problems of Stakeholders 20

Chapter 2: Literature Review 20

2.1 Review of Existing Systems 21

2.2 Comparative Analysis of Techniques/Models 22

Chapter 3: Software Requirement Specification 23

2.1 List of Features 23

2.2 Functional Requirements 24

2.3 Non-Functional Requirements 25

2.4 Use Cases/ Use Case Diagram 25

2.5 Software Development Plan 27

Chapter 4 System Overview 28

3.1 Architectural Design 28

3.2 Data Design 29

3.2.1 Entity-Relationship Diagram (ERD) 30

3.2.2 Data Flow Diagram (DFD) 36

3.3 Domain Model 40

3.3.1 Class Diagram 40

3.4.1 Activity Diagram 42

3.4.2 Class Diagram 43

3.4.3 Class-level Sequence Diagram 45

3.4.4 State Transition Diagram 47

3.4.5 Activity Diagram 48
3.4.6 Sequence Diagram 49
3.4.7 Component Diagram 51
3.4.8 System Architecture 54
3.4.9 Navigation Flow 58
Chapter 5: System Implementation 63
4.1 Technology Stack (Languages, Frameworks, Tools) 63
4.2 Development Environment Setup 65
4.3 System Modules and Components 66
4.4 Database Implementation 68
4.5 APIs and External Integrations (if applicable) 69
4.6 Dataset Selection and Preprocessing 70
4.7 Model Selection and Training 70
4.8 Model Evaluation 70
4.9 Model Deployment 70
Chapter 6: System Testing & Deployment 71
5.1 Testing Approach (Manual, Automated) 72
5.2 Unit Testing 74
5.3 Integration Testing 76
5.4 System Testing 82
5.5 User Acceptance Testing (UAT) 87
5.6 Performance Testing 92
5.7 Deployment Strategy 95
5.8 Hosting & Deployment Environment 96
5.9 Version Control and CI/CD Pipelines 97
Chapter 7: Results and Discussion 98
6.1 Evaluation and Results 98
6.1.1 Comparison with Initial Objectives 99
6.1.2 User Feedback and Usability Testing 99
6.1.3 Performance Evaluation 99
6.1.4 Challenges and Limitations 100
Chapter 8: Conclusion and Future Work 100
7.1 Conclusion 100
7.2 Future Enhancements 101
Bibliography 102

List of Tables

Table No.	Table Title	Page
Table 1.1	Stakeholders Summary	19
Table 2.1	Comparative Analysis of Existing System Categories	22
Table 4.1	Summary of Major Relationships (Domain Model)	32
Table 4.2	Description of Domain Model Entities and Relationships	34
Table 6.1	Manual Testing – IIS Deployment Issues and Resolutions	72
Table 6.2	Unit Test Report	75
Table 6.3	Integration Testing Tools Used	76
Table 6.4	Integration Test Cases (IT-01 to IT-05)	78
Table 6.5	Key Error Encountered	81
Table 6.6	Integration Testing Final Outcome	81
Table 6.7	System Test Cases (ST-01 to ST-15)	83
Table 6.8	System Testing Summary	87
Table 6.9	User Acceptance Testing (UAT-01 to UAT-15)	89
Table 6.10	Performance Testing Environment	93
Table 6.11	Test Scenarios Executed	93
Table 6.12	Key Performance Metrics Summary	94
Table 6.13	Hosting & Deployment Environment	96
Table 6.14	Development & Deployment Tools Used	96
Table 6.15	Deployment Workflow	97
Table 7.1	Key Findings – Evaluation Results Summary	99

List of Figures

Figure No.	Figure Title	Page
Figure 4.1	EMAP System Overview – Box & Line Context Diagram	28
Figure 4.2	Entity-Relationship Diagram – Group 1 of 3: Core Entities	30
Figure 4.3	Entity-Relationship Diagram – Group 2 of 3: Submission Workflow	30
Figure 4.4	Entity-Relationship Diagram – Group 3 of 3: Evaluation & Archive	31
Figure 4.5	Complete Entity-Relationship Diagram (ERD)	32
Figure 4.6	Data Flow Diagram (DFD) – Level 0: Context Diagram	37
Figure 4.7	Data Flow Diagram (DFD) – Level 1: Main Processes	39
Figure 4.8	Domain Model Class Diagram	41
Figure 4.9	Activity Diagram	43
Figure 4.10	Detailed Class Diagram (Interaction Focus)	44
Figure 4.11	Sequence Diagram – Class-Level: Student & Admin	46
Figure 4.12	State Diagram – ProposalSubmission Lifecycle	48
Figure 4.13	State Diagram – FYP Call Lifecycle (Admin)	49
Figure 4.14	Sequence Diagram – Supervisor Selection Workflow	50
Figure 4.15	Component Diagram – Modules & Dependencies	52
Figure 4.16	Component Diagram – Client–Server Architecture	53
Figure 4.17	Layered MVC Client–Server Architecture Diagram	55
Figure 4.18	Final Master Architecture (FYP Summary)	57
Figure 4.19	Navigation Flow Diagram	59
Figure 4.20	EMAP System Screenshot – User Interface View	60
Figure 4.21	EMAP System Screenshot – User Interface View	62
Figure 4.22	EMAP System Screenshot – User Interface View	62
Figure 4.23	EMAP System Screenshot – User Interface View	62
Figure 6.1	IIS Deployment Error Screenshot	71

Figure 6.2	Testing Approach Overview	72
Figure 6.3	Test Report Screenshot	88
Figure 6.4	Test Report Screenshot	92
Figure 6.5	Deployment Strategy Screenshot	95
Figure 6.6	CI/CD Pipeline Screenshot	98
Figure 7.1	Evaluation Results Summary Chart	99
Figure 7.2	Comparison with Initial Objectives Chart	100
Figure 7.3	User Feedback and Usability Testing Results	102
Figure 7.4	Performance Evaluation Results	102
Figure 7.5	Challenges and Limitations Summary	104
Figure 7.6	EMAP Result Summary	104

List of Acronyms

Acronym	Full Form
API	Application Programming Interface
ASP	Active Server Pages
CI/CD	Continuous Integration / Continuous Deployment
CMS	Content Management System
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DOST	Department of Science and Technology
EMAP	Editorial Management and Automation Platform
ERD	Entity-Relationship Diagram
FYP	Final Year Project
HED	Higher Education Department
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IIS	Internet Information Services
LINQ	Language Integrated Query
LMS	Learning Management System
MSSQL	Microsoft SQL Server
MVC	Model-View-Controller
ORM	Object-Relational Mapper
PDF	Portable Document Format
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
SaaS	Software as a Service
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRS	Software Requirements Specification
SSMS	SQL Server Management Studio
SSL/TLS	Secure Sockets Layer / Transport Layer Security

UAT	User Acceptance Testing
UI	User Interface
WCAG	Web Content Accessibility Guidelines

Chapter 1: Introduction

1.1 Problem Statement

Educational institutions, universities, and research organizations often rely on fragmented, manual, or semi-digital systems to manage Final Year Projects (FYP), thesis workflows, and research publication processes. These traditional methods create major challenges, including delayed approvals, inefficient communication between students and supervisors, poor tracking of submissions, inconsistent reviewer assignment, limited transparency, and difficulties in maintaining centralized academic records. Manual handling increases administrative workload, introduces errors, and reduces overall institutional efficiency. Additionally, the absence of a unified system for FYP, thesis, and research management limits scalability and makes it difficult for institutions to maintain quality assurance. Therefore, there is a strong need for an integrated digital platform that automates and centralizes these academic workflows.

1.2 Business Opportunity

The growing demand for digital transformation in higher education presents a significant opportunity for a centralized academic workflow automation platform. EMAP (Editorial Management and Automation Platform) addresses a critical institutional gap by providing an all-in-one solution for managing FYPs, thesis submissions, research publications, reviewer coordination, and academic reporting. Universities, colleges, Higher Education Departments (HED), and research organizations can benefit from streamlined operations, improved transparency, and enhanced productivity. The platform also offers scalability for commercialization as a Software-as-a-Service (SaaS) solution for educational institutions seeking modernization.

1.3 Objectives

The primary objectives of EMAP are:

- To automate Final Year Project, thesis, and research management workflows.
- To centralize academic submission, approval, and evaluation processes.
- To improve communication between students, supervisors, reviewers, coordinators, and administrators.
- To provide role-based dashboards for secure and efficient user interaction.
- To reduce paperwork, delays, and manual administrative effort.
- To ensure transparency, accountability, and auditability in academic workflows.

- To generate reports, analytics, and archival records for institutional decision-making.
- To create a scalable system adaptable for universities, HED, DOST, and research bodies.

1.4 Project Scope

EMAP is designed to cover the following areas:

Included Scope:

- FYP lifecycle management (group creation, proposal, defense, chapters, evaluations, finalization)
- Thesis submission and approval workflow
- Research paper/editorial workflow
- Role-Based Access Control (RBAC)
- Automated notifications and communication
- Reporting and analytics dashboards
- Final academic record archival

Excluded Scope:

- Direct plagiarism detection engine development (integration-ready only)
- Payment gateway systems
- Full LMS replacement
- AI-based reviewer recommendation (future enhancement)
- Blockchain verification systems (future enhancement)

1.5 Constraints

The project is subject to the following constraints:

- Limited academic project duration (FYP-1 and FYP-2 semesters)
- Restricted development team size (3 students)
- Limited budget for enterprise tools and hosting
- Dependence on available institutional requirements
- Technology stack limitations based on team expertise
- Time required for testing, debugging, and deployment
- Potential future scalability challenges

1.6 Feasibility

1.6.1 Technical Feasibility

The project is technically feasible because the required technologies, including ASP.NET/.NET Framework, SQL Server, Bootstrap, GitHub, and Cloudflare deployment tools, are accessible and well-supported. The development team possesses the technical skills necessary to implement the system architecture, RBAC, workflow automation, and deployment.

1.6.2 Resource Feasibility

The project requires minimal financial investment due to the use of accessible development tools, university resources, and cloud-based deployment solutions. Human resources include a three-member development team and one academic supervisor, which is sufficient for successful project completion.

1.6.3 Time Feasibility

The project timeline is feasible within the allocated Final Year Project schedule (FYP-1 and FYP-2), with Chapter 1–4 focused on planning, literature review, and design, and Chapter 5–8 dedicated to implementation, testing, deployment, and finalization.

1.6.4 Market Feasibility

There is strong market potential for EMAP due to the increasing digitalization needs of academic institutions. Universities and educational organizations require centralized systems to improve efficiency, making EMAP a practical and adoptable solution with potential expansion into institutional SaaS deployment.

1.7 Stakeholders Description

The project involves multiple stakeholders who directly or indirectly influence its success:

- Students
- Supervisors
- Coordinators
- Academic Administrators
- Reviewers / Committee Members
- University Departments
- Higher Education Bodies (HED/DOST)
- Development Team

1.7.1 Stakeholders Summary

Table 1.1: Stakeholders Summary

Stakeholder	Description
Students	Primary users who submit proposals, chapters, thesis, and research work.
Supervisors	Guide students, review submissions, and approve academic progress.
Coordinators	Manage FYP workflows, evaluations, rubrics, and scheduling.
Administrators	Oversee institutional-level controls, supervisor directories, final records, and archival systems.
Reviewers / Committees	Evaluate academic work and maintain quality assurance.
University Departments	Use the platform for operational academic governance.
Development Team	Designs, develops, tests, and maintains EMAP.

1.7.2 Key High-Level Goals and Problems of Stakeholders

- **Students**

  Goals: Easy submission, transparent progress tracking, timely approvals.

  Problems: Delays, unclear communication, manual bottlenecks.

- **Supervisors**

  Goals: Efficient student management, proposal/chapter review, workload balance.

  Problems: Fragmented records, communication overload.

- **Coordinators**

  Goals: Workflow control, scheduling, evaluation management.

  Problems: Manual scheduling complexity, inconsistent tracking.

- **Administrators**

  Goals: Institutional oversight, final records, system governance.

  Problems: Record fragmentation, reporting inefficiency.

- **University / HED Bodies**

  Goals: Standardization, transparency, academic quality assurance.

  Problems: Lack of centralized digital infrastructure.

- **Development Team**

  Goals: Build scalable, maintainable, and impactful academic software.

  Problems: Time constraints, resource limitations, evolving requirements.

Chapter 2: Literature Review

This chapter reviews existing editorial, peer-review, and academic project management platforms that informed the design of the Editorial Management and Automation Platform (EMAP). The review draws on a recent systematic mapping study of 73 sources spanning journal management systems (JMS), conference management systems (CMS), Final Year Project (FYP) and thesis management systems, and cross-cutting automation technologies. The aim is to identify the strengths and limitations of prior work and to justify the design decisions adopted in EMAP.

2.1 Review of Existing Systems

Journal and Manuscript Management Systems. Open Journal Systems (OJS), ScholarOne, and Editorial Manager are among the most widely adopted platforms for handling the journal submission-to-publication lifecycle. Comparative studies [38], [39] report that these systems automate submission, reviewer coordination, and editorial decisions effectively, but consistently show usability gaps in learnability and efficiency. The OJS ecosystem dominates open-source deployments and supports plugin-based extensions for analytics, accreditation reporting, and metadata, although recent work [40] documents persistent security vulnerabilities such as cross-site scripting. National-scale deployments such as ACOMS+ [41] demonstrate that API-first design and integrated open peer review are achievable, but these capabilities are rarely combined with student-project workflows in a single platform.

Conference and Peer-Review Management Systems. HotCRP [37] and EasyChair are the most widely used conference management systems, providing reviewer assignment, paper bidding, and conflict-of-interest handling. The Toronto Paper Matching System [36] introduced topic-model based automatic reviewer assignment and remains a baseline against which newer approaches are compared. Privacy-focused systems such as ConfiChair [34] and the formally verified CoCon [42] demonstrate that cryptographic guarantees of submission confidentiality are technically feasible, although their adoption in mainstream platforms remains limited. A systematic literature review [43] of the reviewer assignment problem across 103 studies and 30 years confirms that assignment quality, workload balance, and explainability remain open challenges.

FYP and Thesis Management Systems. Student-project platforms reported in the literature range from lightweight PHP/MySQL portals [20] to dashboard-driven systems with role-based access

control such as those described in [15] and [16]. [14] propose a Theory-of-Shame-based accountability mechanism to improve student engagement. Several studies integrate FYP workflows into Moodle [44], [45]. On the algorithmic side, [46] and [47] apply genetic algorithms to student-supervisor allocation, while [35] formalise the problem as a stable-matching problem with preferences over projects. Most FYP systems, however, are reported as small-scale prototypes evaluated with fewer than thirty users and rarely address analytics, interoperability, or governance reporting.

Cross-Cutting Automation and Integrity Technologies. Recent research applies artificial intelligence to reviewer recommendation and editorial decision support. PEERRec [48] uses natural language processing to predict editorial decisions, while MRGen [49] generates meta-reviews from reviewer comments. [50] provide critical empirical evidence that AI-assisted reviews tend to inflate scores and acceptance rates, motivating disclosure and human-oversight requirements. Process mining on event logs, as demonstrated in [51], enables bottleneck detection in peer-review workflows. Blockchain-based approaches for tamper-evident audit trails have been surveyed extensively [52], but their adoption is constrained by scalability and onboarding costs.

2.2 Comparative Analysis of Techniques/Models

A comparative analysis of the four categories of existing systems reveals a clear pattern. Journal management systems are mature in handling submission and review lifecycles but underdeveloped in analytics and security. Conference management systems excel in reviewer assignment algorithms and, in some cases, formal privacy guarantees, but they do not support student-project workflows. FYP management systems are typically prototype-grade and lack interoperability with scholarly publishing infrastructure such as ORCID, DOI, and OAI-PMH. Cross-cutting AI and blockchain technologies offer promising capabilities, but require governance frameworks that are absent from most deployed platforms. Table 2.1 summarises these patterns across the four domains.

Table 2.1 Comparative Analysis of Existing System Categories

Dimension	JMS	CMS	FYP	AI / Blockchain
Workflow automation	Mature (OJS)	Mature (HotCRP)	Partial (prototypes)	Emerging
Analytics	Limited	Moderate	Weak dashboards	Strong potential

Security / privacy	Inconsistent	High (formal proofs)	Rarely addressed	Varies widely
Interoperability	OAI-PMH partial	API-first (some)	LMS-dependent	Limited
Evaluation rigor	Usability studies	Algorithm benchmarks	Small prototypes	Mixed evidence

The analysis highlights three gaps that motivate EMAP. First, no existing system unifies journal-style submission, conference-style reviewing, and student-project supervision under a single governance framework; EMAP addresses this by adopting a configurable workflow engine that supports all three workflows. Second, reviewer and supervisor assignment is treated either as an algorithmic problem (in CMS) or as a manual coordination task (in FYP systems), but rarely as an auditable service with logged decision criteria; EMAP implements assignment as a dedicated module with role-based access control and audit logging. Third, security, privacy, and interoperability features such as RBAC, encrypted document storage, ORCID, and DOI integration are inconsistently implemented in the literature; EMAP treats these as first-class platform requirements rather than optional extensions. Together, these design decisions position EMAP as an integrated and auditable platform that addresses the documented limitations of prior work and provides a foundation for the requirements specified in the next chapter.

Chapter 3: Software Requirement Specification

3.1 List of Features

The Editorial Management and Automation Platform (EMAP) provides a comprehensive digital solution to manage academic submissions and reviews.

Key features include:

- **FYP Management:** Submission, review, approval, and grading of final-year projects.
- **Thesis Management:** Online submission, plagiarism verification, and reviewer evaluation.
- **Research Management:** Handling of research paper submissions and peer-review workflows.
- **Role-Based Dashboards:** Customized interfaces for students, supervisors, reviewers, and administrators.
- **Automated Notifications:** Real-time alerts via email and dashboard for deadlines and feedback.
- **Analytics & Reporting:** Data visualization and export options for decision-making.
- **Secure User Management:** Authentication and authorization through RBAC.
- **Integration Tools:** Connection with plagiarism detection and email systems.
- **Centralized Data Storage:** Safe archival and retrieval of academic records.
- **Backup & Recovery:** Daily automated backups with restoration options.

3.2 Functional Requirements

The EMAP system provides the following major functional capabilities:

FYP Management Module

- Allows students to submit FYP proposals and progress reports.
- Enables supervisors to review, approve, or reject submissions.
- Provides automated grading and maintains submission logs.

Thesis Management Module

- Facilitates online thesis submission and plagiarism checking.
- Allows reviewer assignment, feedback submission, and digital approvals.

- Generates plagiarism and evaluation reports.

Research Management Module

- Manages research article submissions and peer review processes.
- Tracks reviewer decisions (accept/reject/revise).
- Handles publication-ready files and archiving.

User & Role Management

- Supports secure login and role-based access control.
- Allows administrators to add, edit, or deactivate users.
- Maintains an audit trail for accountability.

Notification & Communication System

- Sends automated emails and dashboard alerts for updates.
- Maintains communication logs for all users.

Reporting & Analytics

- Generates submission, evaluation, and performance reports.
- Supports data visualization and export in PDF, CSV, and Excel formats.

3.3 Non-Functional Requirements

Performance:

- The system shall process all transactions within **3 seconds**.
- It must support **up to 500 concurrent users** efficiently.

Safety:

- **Daily data backups** must be performed automatically.
- **Data restoration** capability shall ensure recovery in case of system failure.

Security:

- All communications must be encrypted using **SSL/TLS**.
- **Role-Based Access Control (RBAC)** must be implemented for user-level security.

Software Quality Attributes:

- **Usability:** The interface shall be intuitive and mobile-responsive.
- **Reliability:** Ensure **99.5% uptime** and consistent data accuracy.
- **Maintainability:** Modular architecture to allow easy updates and debugging.

- **Scalability:** System should support **multi-institutional expansion** without degradation.

Business Rules:

- Only **authorized users** can perform role-specific actions.
- **Administrators** manage roles, backups, and recovery operations.
-

3.4 Use Cases/ Use Case Diagram

Primary Actors:

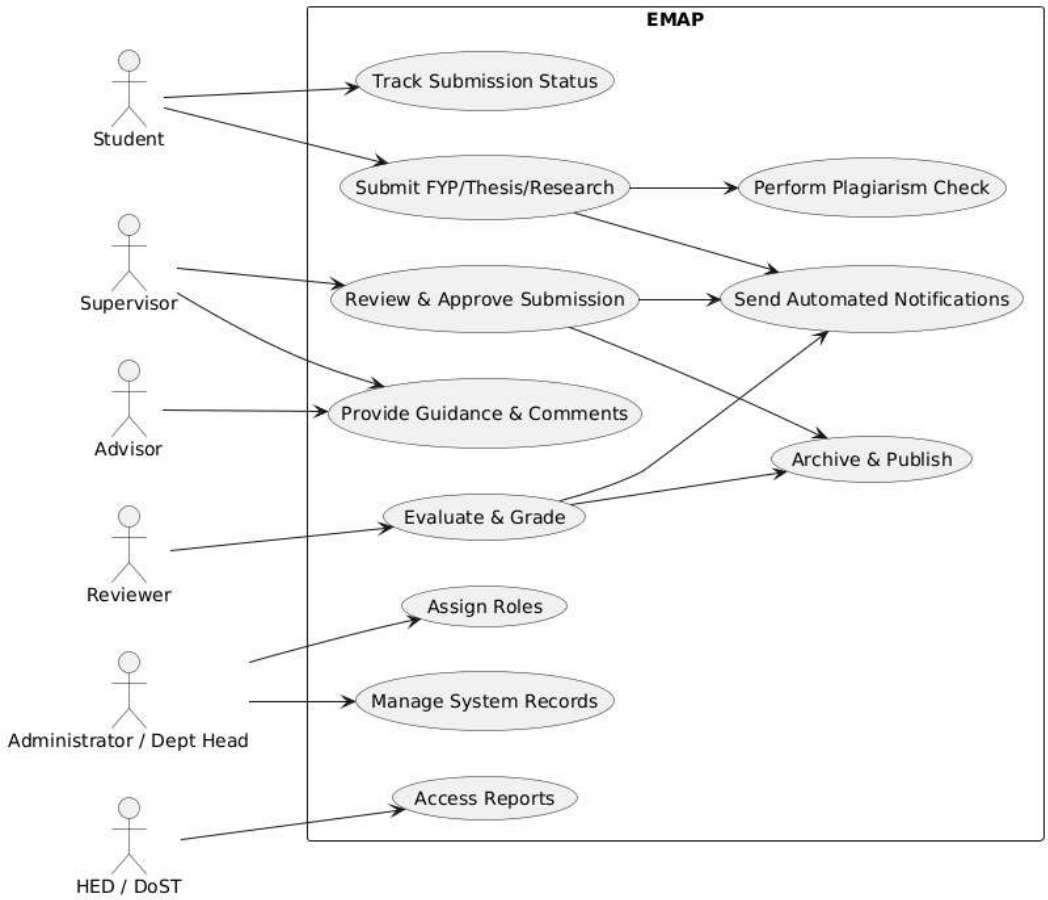
- Students
- Supervisors
- Reviewers
- Administrators

Sample Use Cases:

1. **Submit Thesis/FYP:** Student uploads file → System validates → Supervisor reviews → Admin stores record.
2. **Evaluate Submission:** Supervisor/Reviewer reviews document → Provides feedback → System updates status.
3. **Generate Report:** Administrator selects report type → System compiles and exports in preferred format.
4. **Send Notification:** System triggers automated alerts on submission updates or deadlines.

Use Case Diagram (Text Description):

- **Students** interact with submission and tracking modules.
- **Supervisors and Reviewers** evaluate submissions and provide feedback.
- **Administrators** manage users, roles, reports, and system configurations.



3.5 Software Development Plan

1. **Development Model:** Agile methodology for iterative and feedback-driven progress.
2. **Frontend Technology:** HTML5, CSS3, JavaScript (responsive UI).
3. **Backend Framework: .**NET Framework.
4. **Database:** MySQL or compatible RDBMS.
5. **Web Server:** Apache or Nginx.
6. **Hosting Environment:** Institutional or cloud-based servers.
7. **Version Control:** GitHub for collaboration and source management.
8. **Testing Tools:** Unit and integration testing using .NET testing frameworks.
9. **Deployment:** Final deployment on university or institutional infrastructure.

Chapter 4: System Overview

4.1 Architectural Design

Figure 4.1 presents the emap system overview – box & line context diagram.

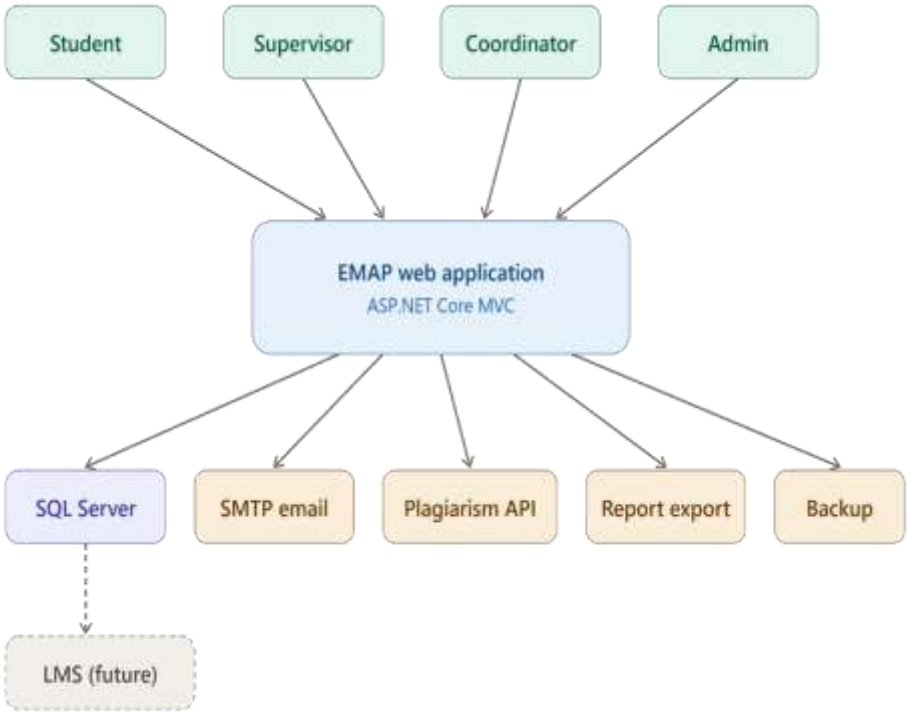


Figure 4.1: EMAP System Overview – Box & Line Context Diagram

EMAP – System Overview (Box & Line Context Diagram)

Purpose:

- To present a high-level overview of the EMAP system, showing major users, system components, and external integrations.

Description:

- The diagram places the **EMAP Web Application** at the center, implemented using **ASP.NET Core MVC and Bootstrap**.
- Different user roles—**Student, Supervisor, Reviewer, and Administrator**—interact with the system based on their responsibilities.
- Students submit and track FYP, thesis, and research work.

- Supervisors and reviewers review, evaluate, and provide feedback on submissions.
- Administrators manage users, roles, reports, and system configuration.
- The system connects to external services such as database storage, email notifications, plagiarism checking, reporting, and backup services.

Key Highlights:

- Represents a **simple Box & Line view** suitable for initial system design.
- Shows **centralized data storage** using SQL Server.
- Illustrates integration with **external services** (Email, Plagiarism Tool, Reporting, Backup).
- Helps non-technical stakeholders understand system boundaries and interactions.

4.2 Data Design

4.2.1 ERD Group 1 of 3 — Core entities (Users, FYP calls, groups)

Figure 4.2 presents the entity-relationship diagram – group 1 of 3: core entities.

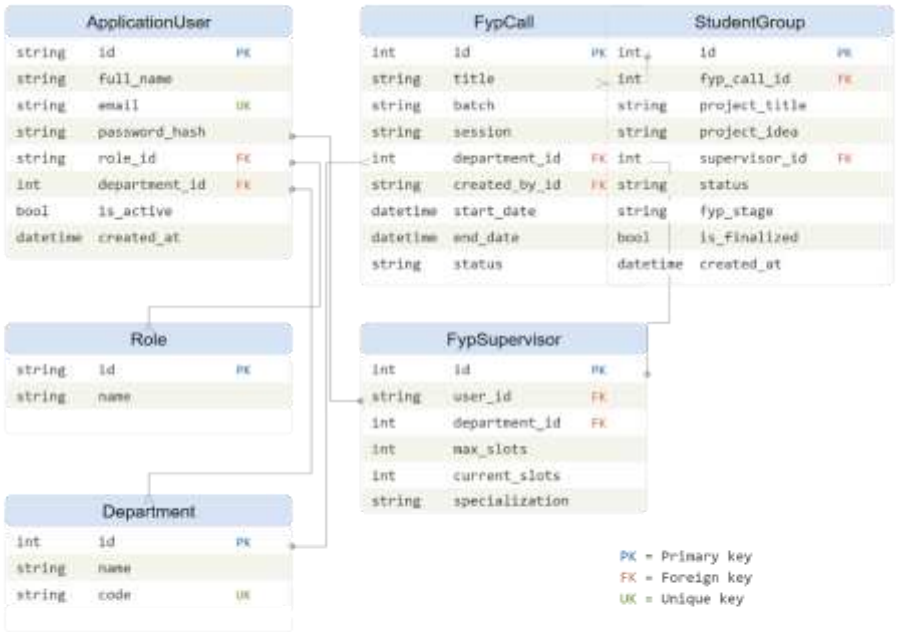


Figure 4.2: Entity-Relationship Diagram – Group 1 of 3: Core Entities

4.2.2 ERD Group 2 of 3 - Submission workflow (Proposal, Defense, Chapters)

Figure 4.3 presents the entity-relationship diagram – group 2 of 3: submission workflow.

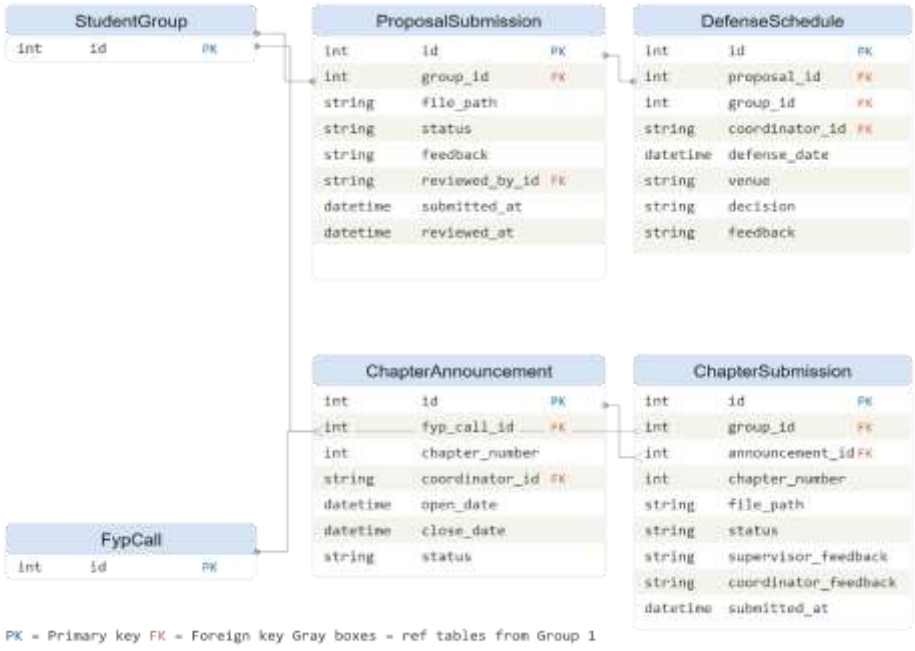


Figure 4.3: Entity-Relationship Diagram – Group 2 of 3: Submission Workflow

4.2.3 ERD Group 3 of 3 — Evaluation & Archive

Figure 4.4 presents the entity-relationship diagram – group 3 of 3: evaluation & archive.

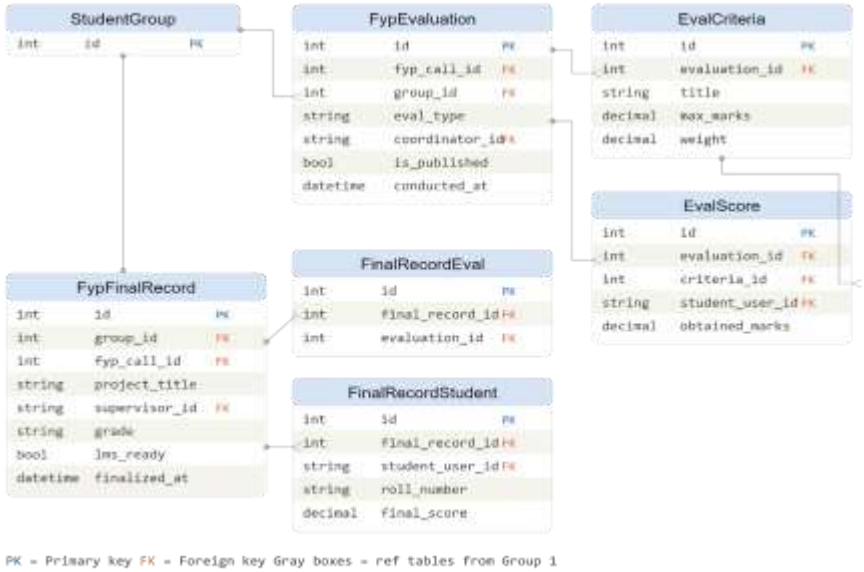


Figure 4.4: Entity-Relationship Diagram – Group 3 of 3: Evaluation & Archive

4.2.4 Entity-Relationship Diagram (ERD)

Figure 4.5 presents the complete entity-relationship diagram (erd).

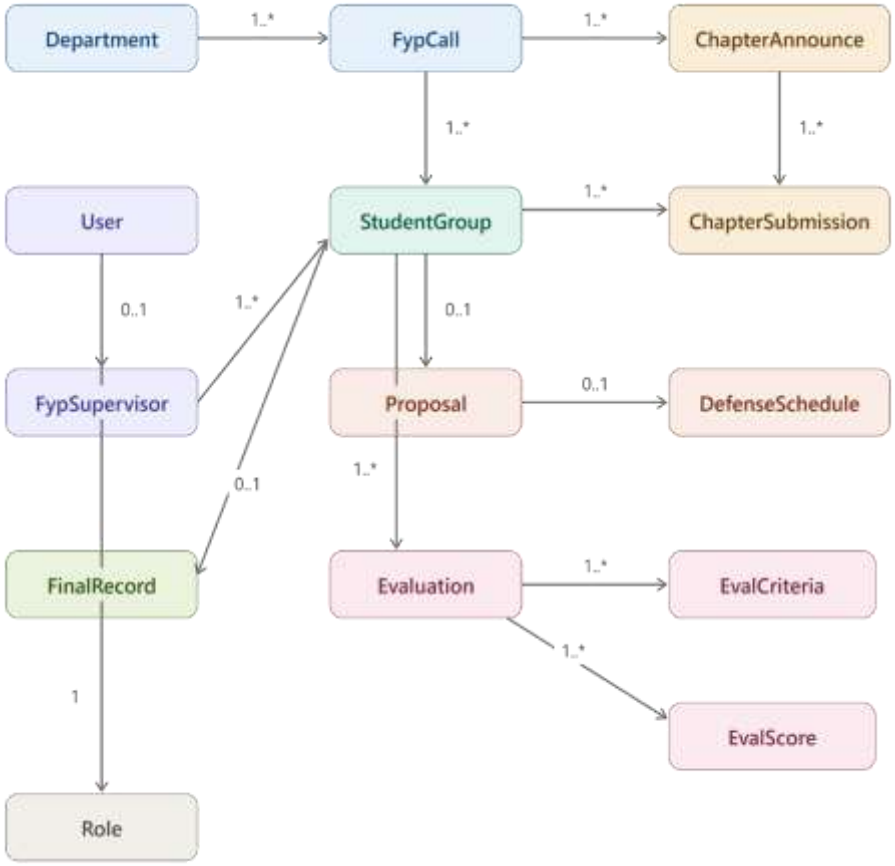


Figure 4.5: Complete Entity-Relationship Diagram (ERD)

4.2.5 Summary of Major Relationships

Table 4.1 below summarizes summary of major relationships (domain model)table 4.1: summary of major relationships (domain model)table 4.1: summary of major relationships (domain model).

Table 4.1: Summary of Major Relationships (Domain Model)

RELATIONSHIP	CARDINALITY	DESCRIPTION
DEPARTMENT → FYPCALL	1 : Many	A department can create multiple FYP calls.
FYPCALL → STUDENTGROUP	1 : Many	A single FYP call can involve multiple student groups.
FYPCALL → CHAPTERANNOUNCE	1 : Many	Multiple chapter announcements can be issued under one FYP call.
CHAPTERANNOUNCE → CHAPTERSUBMISSION	1 : Many	One announcement can receive multiple chapter submissions.
FYPSUPERVISOR → STUDENTGROUP	1 : Many	A supervisor can supervise multiple student groups.
STUDENTGROUP → PROPOSAL	1 : 0..1	A student group may submit one proposal.
PROPOSAL → DEFENSESCHEDULE	1 : 0..1	A proposal may have one defense schedule.
PROPOSAL → EVALUATION	1 : Many	A proposal can undergo multiple evaluations.
EVALUATION → EVALCRITERIA	1 : Many	An evaluation uses multiple assessment criteria.
EVALUATION → EVALSCORE	1 : Many	Multiple scores are generated during an evaluation.

4.2.6 Description of Domain Model Entities and Relationships

Table 4.2 below summarizes description of domain model entities and relationshipstable 4.2: description of domain model entities and relationshipstable 4.2: description of domain model entities and relationships.

Table 4.2: Description of Domain Model Entities and Relationships

Entity	Description	Primary Responsibilities	Key Relationships
Department	Represents an academic department responsible for managing Final Year Project activities.	• Manage FYP processes • Organize project calls • Provide administrative oversight	One Department can manage multiple **FypCalls**.
FypCall	Represents an official call for Final Year Project submissions issued by a department.	• Define project submission periods • Publish deadlines • Coordinate project activities	Belongs to one **Department** and can involve multiple **StudentGroups** and **ChapterAnnouncements**.
User	Represents any individual who accesses the system, including students, supervisors, and administrators.	• User authentication • Account management • System access	Associated with **Role** and may act as a **FypSupervisor**.
Role	Defines user	• Access control	One Role can be assigned

	permissions and access levels within the system.	• Authorization management • Permission assignment	to multiple **Users**.
FypSupervisor	Represents academic staff responsible for supervising student projects.	• Guide student groups • Monitor progress • Review project work	Derived from a **User** and can supervise multiple **StudentGroups**.
StudentGroup	Represents a group of students collaborating on a Final Year Project.	• Submit proposals • Upload chapters • Participate in evaluations	Associated with a **FypCall**, supervised by a **FypSupervisor**, and linked to **Proposal** and **ChapterSubmission** entities.
Proposal	Represents a project proposal submitted by a student group.	• Store project details • Record proposal status • Support evaluation process	Submitted by a **StudentGroup**, evaluated through **Evaluation**, and may have a **DefenseSchedule**.
DefenseSchedule	Stores information related to proposal defense presentations.	• Schedule defenses • Manage presentation sessions • Support evaluation activities	Linked to a specific **Proposal**.
Evaluation	Represents the assessment conducted for a project proposal.	• Record evaluation outcomes • Store feedback • Manage assessment records	Associated with a **Proposal**, **EvalCriteria**, and **EvalScore**.

EvalCriteria	Defines the criteria used to assess project proposals.	• Establish assessment standards • Standardize evaluation process	One Evaluation can use multiple **EvalCriteria**.
EvalScore	Stores the scores awarded during an evaluation.	• Record marks • Calculate evaluation results • Maintain detailed scoring records	Generated from an **Evaluation** and linked to specific evaluation criteria.
ChapterAnnounce	Represents chapter submission announcements issued during the project lifecycle.	• Publish chapter deadlines • Provide submission instructions • Communicate requirements	Belongs to an **FypCall** and can have multiple **ChapterSubmissions**.
ChapterSubmission	Represents chapter documents submitted by student groups.	• Submit project chapters • Track progress • Maintain submission records	Submitted by a **StudentGroup** in response to a **ChapterAnnounce**.
FinalRecord	Stores final project information and completion records.	• Record project completion • Store final outcomes • Maintain academic records	Associated with supervised student projects and final project results.

4.2.7 Data Flow Diagram (DFD)

EMAP – Data Flow Diagram (DFD Level 0: Context Diagram)

Figure 4.6 presents the data flow diagram (dfd) – level 0: context diagram.

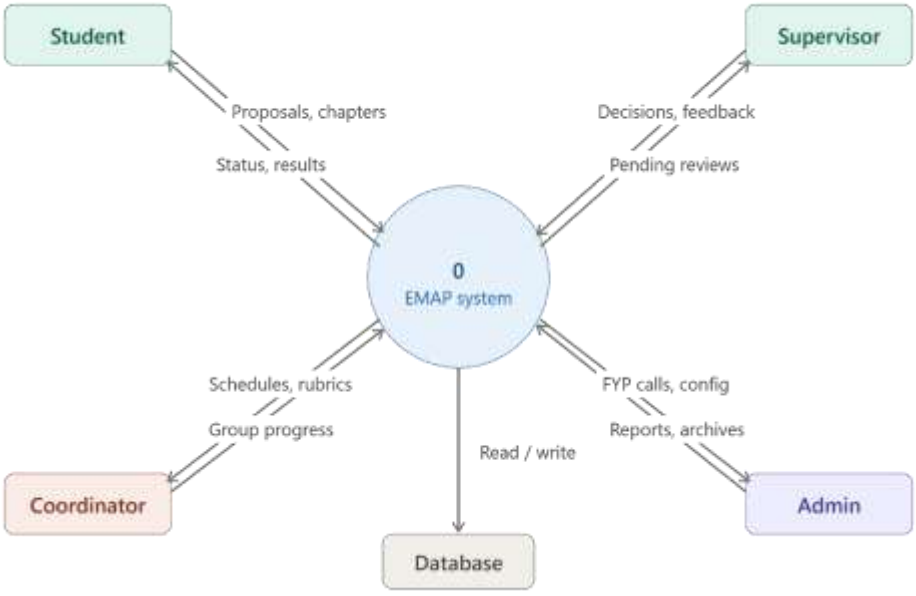


Figure 4.6: Data Flow Diagram (DFD) – Level 0: Context Diagram

Purpose:

- To provide a high-level view of the EMAP system and its interaction with external users.

Description:

- The diagram represents **EMAP as a single central process**.
- **Students** submit proposals and view their proposal status through the system.
- **Supervisors** review proposals and perform approval or rejection actions.
- **Administrators** create FYP calls and manage system users.
- The EMAP system stores and retrieves users, calls, proposals, and project data from the **EMAP Database**.

Key Highlights:

- Shows system boundaries clearly using a single process.
- Illustrates interaction between users and the system without internal details.
- Represents centralized data storage through the EMAP database.
- Serves as the foundation for the detailed Level 1 DFD.

EMAP – Data Flow Diagram (DFD Level 1: Main Processes)

Figure 4.7 presents the data flow diagram (dfd) – level 1: main processes.

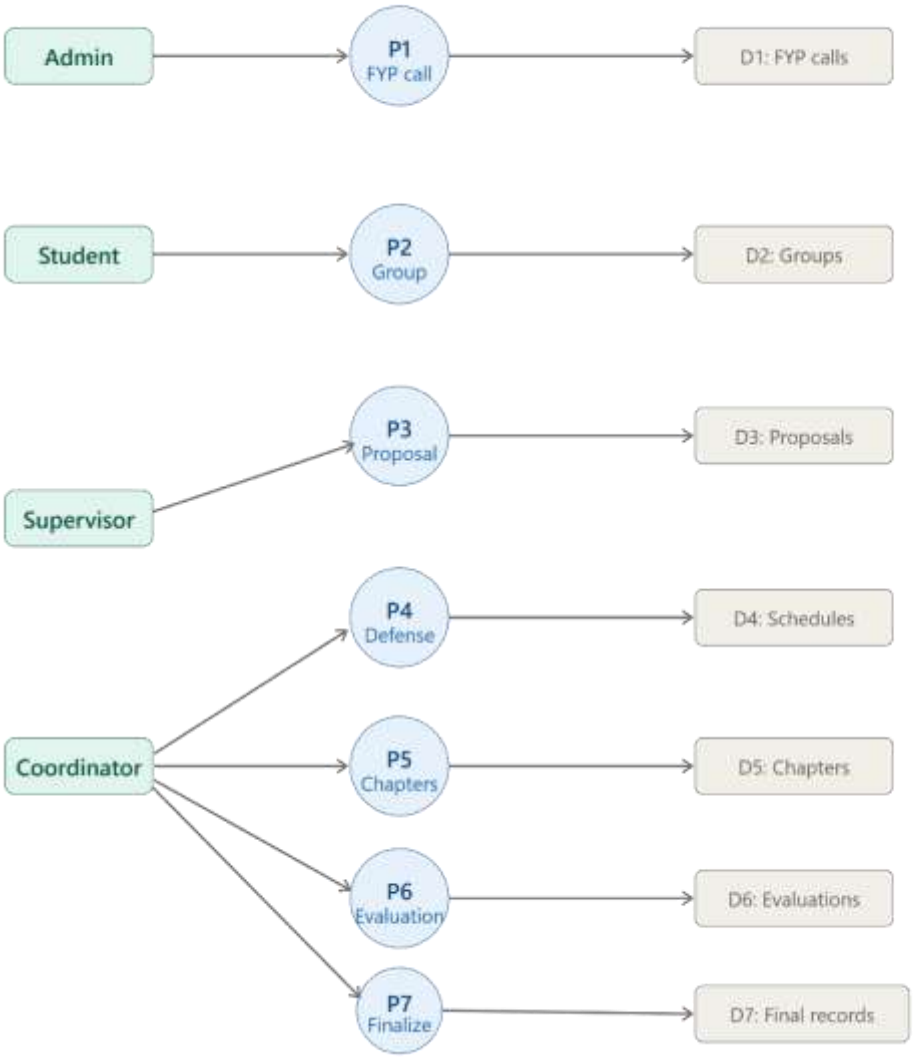


Figure 4.7: Data Flow Diagram (DFD) – Level 1: Main Processes

Purpose:

- To show the internal data flow between major processes, users, and data stores in the EMAP system.

Description:

- The diagram decomposes the system into four main processes: **Manage FYP Call**, **Submit Proposal**, **Review Proposal**, and **Create/Assign Project**.

- The **Admin** interacts with *P1: Manage FYP Call* to create or update FYP calls, which are saved in the **FYP Calls** data store.
- The **Student** uses *P2: Submit Proposal* to upload proposals and select an FYP call, with proposal data stored in the **Proposals** data store.
- The **Supervisor** interacts with *P3: Review Proposal* to view and decide on submitted proposals, updating their status as approved or rejected.
- The system uses user and role information from the **Users & Roles** data store to validate permissions.
- Approved proposals are processed in *P4: Create/Assign Project*, where project records are created and stored in the **Projects** data store.

Key Highlights:

- Clearly separates processes from data stores using standard DFD notation.
- Demonstrates how data flows from users to processes and then to databases.
- Shows role-based validation using the Users & Roles data store.
- Provides a detailed view of system functionality beyond the context diagram.

4.3 Domain Model

4.3.1 Class Diagram

Figure 4.8 presents the domain model class diagram.

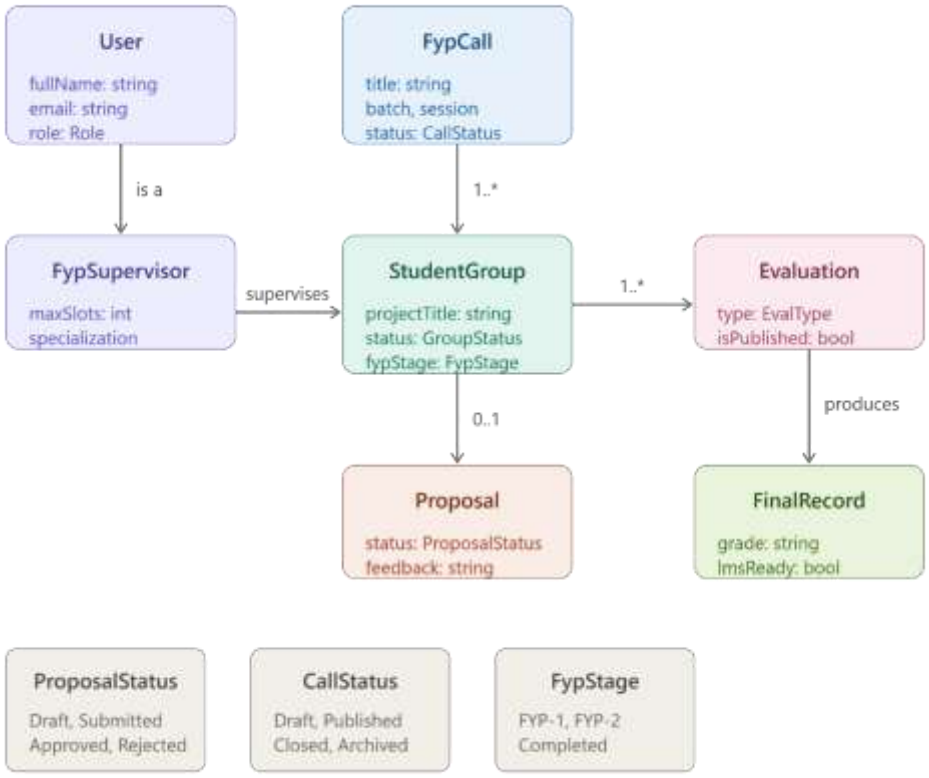


Figure 4.8: Domain Model Class Diagram

EMAP – Domain Model (Class Diagram)

Purpose:

- To represent the core business entities of the EMAP system and their relationships at a conceptual level.

Description:

- The diagram shows main domain entities such as **ApplicationUser, Role, StudentGroup, Supervisor, FypCall, FypProject, and ProposalSubmission**.
- **ApplicationUser** represents system users and is associated with **Role** for role-based access.
- **Supervisor** is a specialized type of ApplicationUser with additional attributes.
- **Students** are organized into **StudentGroups**, which submit **ProposalSubmissions**.
- **Supervisors** review proposals and supervise **FYP Projects**.

- **FYP Calls** define the context under which projects are created.
- Enumerations **ProposalStatus** and **ProjectStatus** define valid states for proposals and projects.

Key Highlights:

- Clearly defines entity relationships such as *submits, reviews, supervises, owns,* and *created under*.
- Shows **one-to-many and many-to-one cardinalities** between entities.
- Uses enumerations to enforce controlled status values.
- Serves as a bridge between system requirements and design implementation.

3.4.1 Activity Diagram

Figure 4.9 presents the activity diagram.

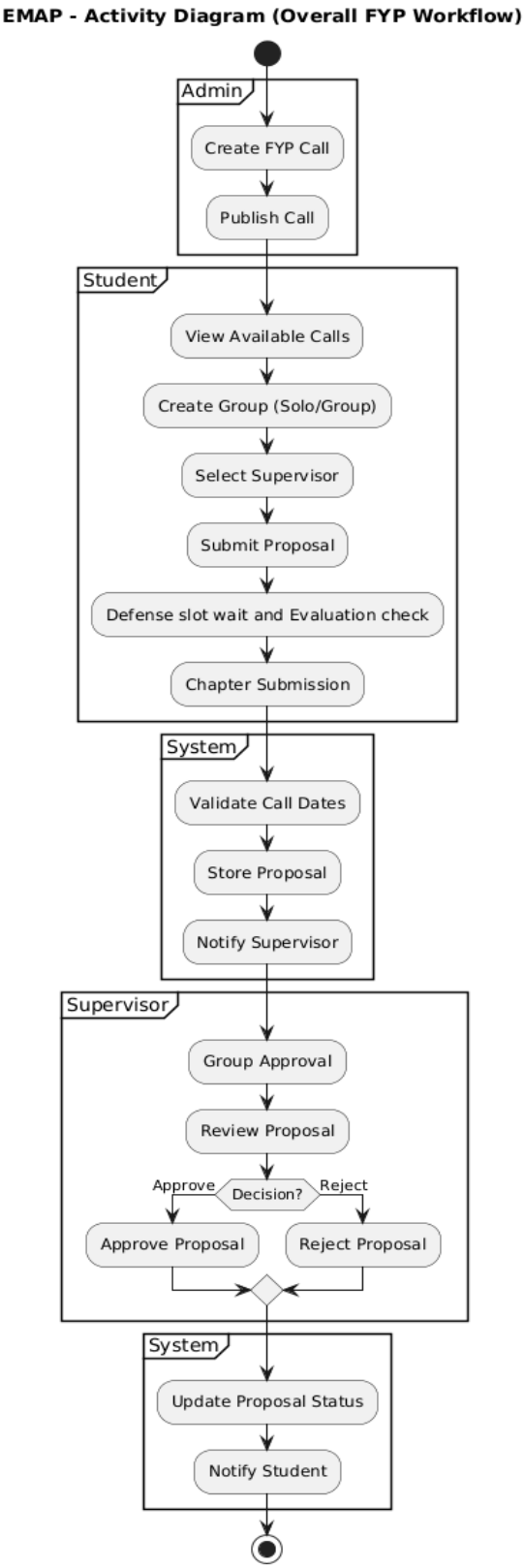


Figure 4.9: Activity Diagram

3.4.2 Class Diagram

EMAP – Detailed Class Diagram (Interaction Focus)

Figure 4.10 presents the detailed class diagram (interaction focus).

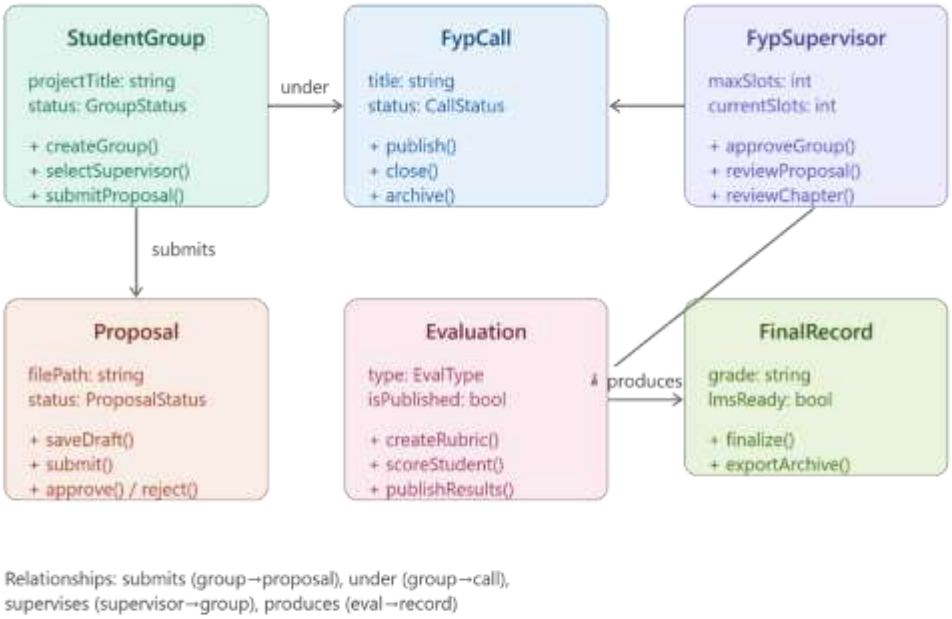


Figure 4.10: Detailed Class Diagram (Interaction Focus)

Purpose:

To illustrate the main system classes along with their attributes, methods, and interactions.

Description:

- The diagram shows core classes such as **ApplicationUser, Supervisor, StudentGroup, FypCall, and ProposalSubmission**.
- **Supervisor** is a specialized type of **ApplicationUser** with proposal

review functionality.

- **StudentGroup** creates and submits proposals.
- **FypCall** defines the timeframe in which proposals are accepted.
- **ProposalSubmission** contains operations such as saving drafts, submitting, approving, and rejecting proposals.
- **ProposalStatus** enumeration defines the states of a proposal.

Key Highlights:

- Includes **methods** to show interaction behavior, not just data structure.
- Clearly shows **one-to-many relationships** between supervisors and proposals.
- Demonstrates proposal lifecycle through class operations.
- Supports interaction-based system design and implementation.

3.4.3 Class-level Sequence Diagram

Figure 4.11 presents the sequence diagram – class-level: student & admin.

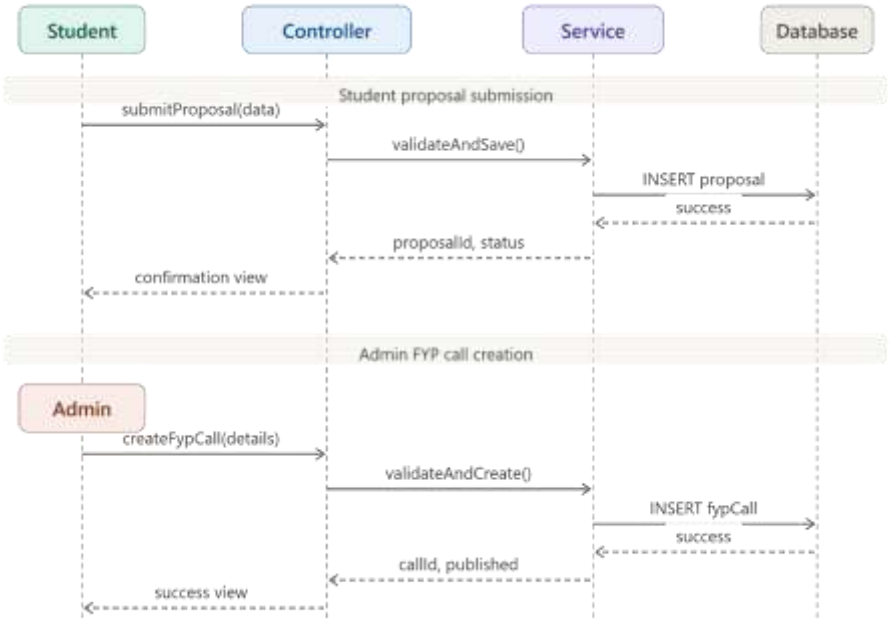


Figure 4.11: Sequence Diagram – Class-Level: Student & Admin

EMAP - Sequence Diagram (Class-Level: Student + Admin)

Purpose:

- To show the sequence of interactions between users, the EMAP system, and the database at the class/service level.

Description:

- The diagram illustrates two main scenarios: **student proposal submission** and **admin FYP call creation**.
- In the first scenario, the **student submits a proposal**, which is received by the EMAP system.
- The system **validates call dates**, stores the proposal in the **SQL Server database**, and returns a submission confirmation with status.
- In the second scenario, the **admin creates an FYP call** by sending request details to the system.
- The system stores the FYP call in the database and confirms successful publication.

Key Highlights:

- Shows **time-ordered message flow** between actors and system components.
- Demonstrates interaction between **controllers/services and the database**.
- Clearly separates student and admin workflows.
- Focuses on **system behavior**, not user interface actions.

3.4.4 State Transition Diagram

EMAP - State Diagram (ProposalSubmission)

Figure 4.12 presents the state diagram – proposalsubmission lifecycle.

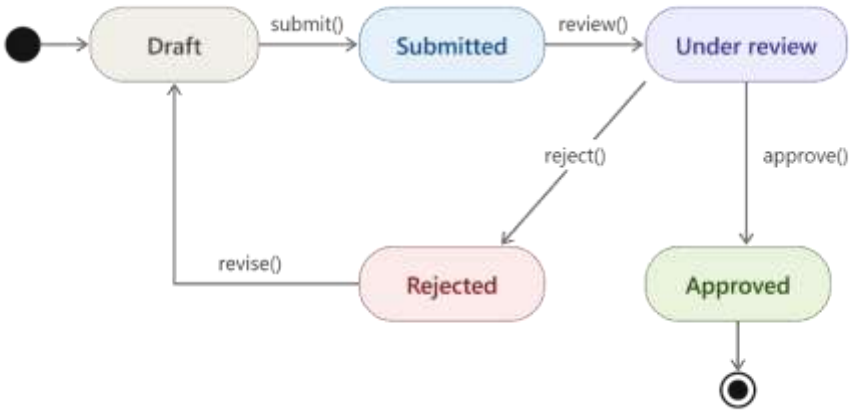


Figure 4.12: State Diagram – ProposalSubmission Lifecycle

Purpose:

- To illustrate the different states of a proposal and the transitions between those states in the EMAP system.

Description:

- The proposal initially starts in the **Draft** state.
- When the student submits the proposal, it moves to the **Submitted** state.
- From the submitted state, the proposal can be **Approved** or **Rejected** based on supervisor decision.
- If rejected, the proposal can be **revised** and returned to the Draft state.
- An approved proposal reaches the final state, indicating completion of the proposal process.

Key Highlights:

- Clearly represents the **proposal lifecycle**.
- Shows valid state transitions using labeled events such as *submit()*, *approve()*, and *reject()*.
- Prevents invalid state changes by defining controlled transitions.
- Useful for enforcing workflow rules in the system.

EMAP – State Diagram (FYP Call Lifecycle by Admin)

Figure 4.13 presents the state diagram – fyp call lifecycle (admin).

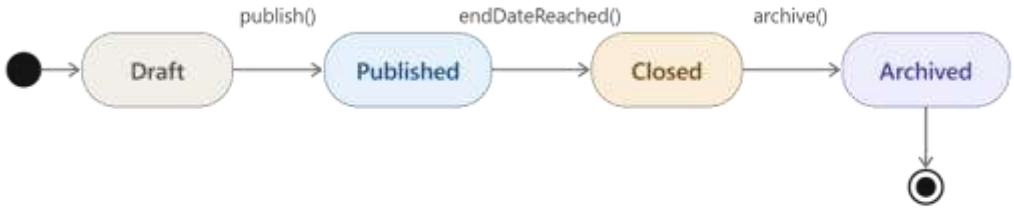


Figure 4.13: State Diagram – FYP Call Lifecycle (Admin)

Purpose:

- To represent the lifecycle of an FYP call and the state changes controlled by the admin.

Description:

- The FYP call starts in the **Draft** state when it is created.
- Once the admin publishes the call, it moves to the **Published** state.
- When the submission end date is reached, the call transitions to the **Closed** state.
- The admin can then archive the call, moving it to the **Archived** state.
- The process ends after the call is archived.

Key Highlights:

- Shows a **linear and controlled lifecycle** of an FYP call.
- Uses event-based transitions such as *publish()*, *endDateReached()*, and *archive()*.

- Prevents invalid operations by restricting state changes.
- Useful for enforcing administrative workflow rules.

3.4.5 Sequence Diagram

EMAP – Sequence Diagram (Class-Level: Student + Admin)

Figure 4.14 presents the sequence diagram – supervisor selection workflow.

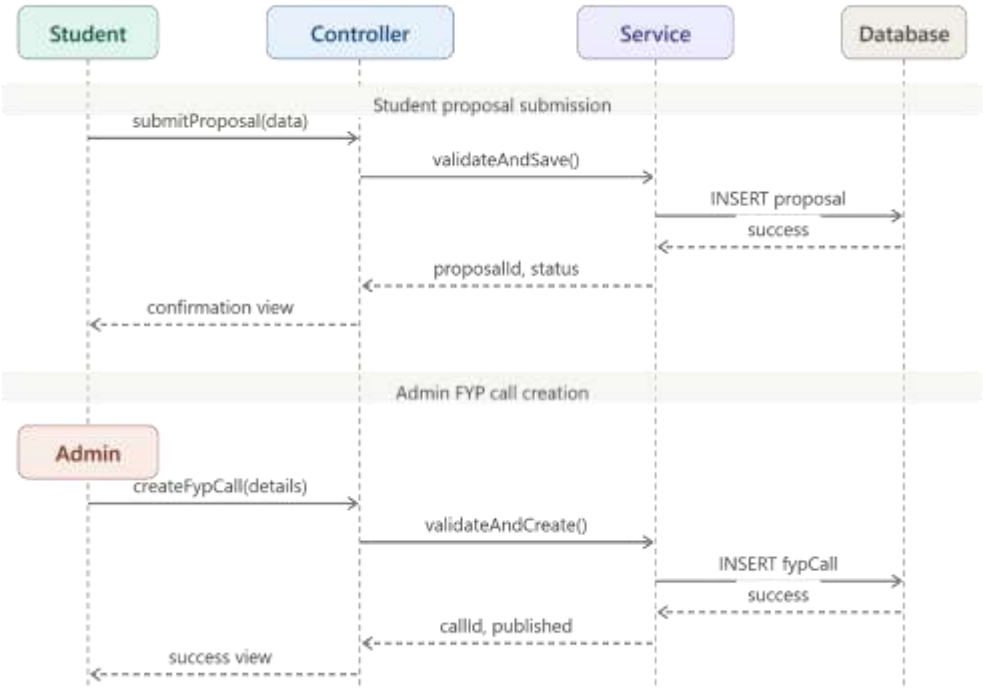


Figure 4.14: Sequence Diagram – Supervisor Selection Workflow

Purpose:

- To illustrate the sequence of interactions between users, the EMAP system, and the database at the class/service level.

Description:

- The diagram shows two main interaction scenarios: **student proposal submission** and **admin FYP call creation**.
- In the first scenario, the **student submits a proposal** to the EMAP system.
- The system **validates call dates**, stores the proposal in the **SQL Server database**, and sends a confirmation with submission status.
- In the second scenario, the **admin creates an FYP call** by providing call details to the system.
- The system stores the FYP call in the database and confirms successful publication.

Key Highlights:

- Displays **time-ordered message flow** between actors and system components.
- Clearly shows interaction between **controllers/services and database**.
- Separates student and admin operations within the same sequence diagram.
- Focuses on **system behavior and data processing**, not UI design.

3.4.6 Component Diagram

EMAP - Component Diagram (Modules & Dependencies)

Figure 4.15 presents the component diagram – modules & dependencies.

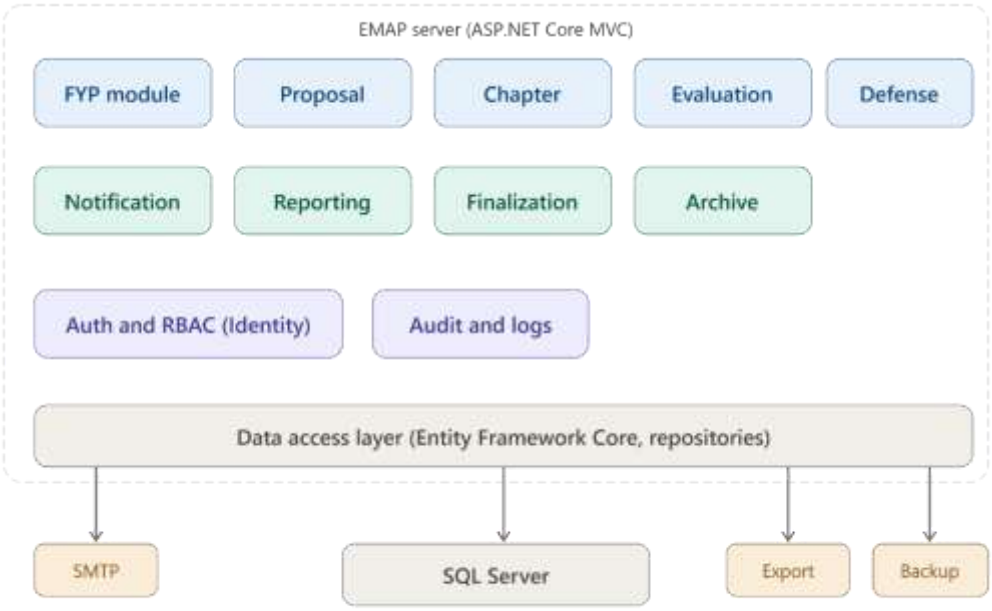


Figure 4.15: Component Diagram – Modules & Dependencies

Purpose:

- To show the major software modules of the EMAP system and how they depend on each other and external services.

Description:

- The diagram represents EMAP as a server-side application built using **ASP.NET Core MVC**.
- Core functional modules include **FYP Module, Thesis Module, Research Module, Reporting & Analytics**, and **Notification Module**.
- **Auth & RBAC** controls authorization across all modules.
- **Audit & Logs** tracks user actions for monitoring and accountability.
- All modules interact with the **Data Access layer** implemented using **Entity Framework Core and repositories**.
- The Data Access layer communicates with the **SQL Server database**.
- External services such as **Email Service, Plagiarism Tool, Reporting Export**, and **Backup Storage** are integrated with relevant modules.

Key Highlights:

- Clearly shows **modular decomposition** of the system.
- Demonstrates **centralized data access** through a common data layer.
- Highlights cross-cutting concerns like **authentication, auditing, and notifications**.
- Illustrates integration with **external systems** for extended functionality

EMAP – Component Diagram (Client–Server)

Figure 4.16 presents the component diagram – client–server architecture.

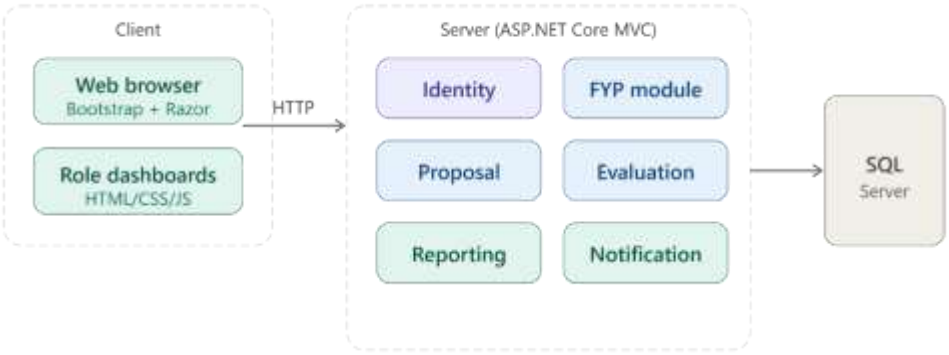


Figure 4.16: Component Diagram – Client–Server Architecture

Purpose:

- To illustrate the client–server architecture of the EMAP system and the interaction between the user interface, server modules, and database.

Description:

- The **client side** consists of a web-based user interface developed using **Bootstrap**.
- The **server side** is implemented using **ASP.NET Core MVC**.
- The **Identity Module** handles authentication and role-based access control.
- The **FYP Module** manages FYP calls and project information.
- The **Proposal Module** supports proposal submission and review activities.
- The **Reporting Module** generates system reports.
- All server modules interact with the **SQL Server database** for data storage and retrieval.

Key Highlights:

- Clearly separates **client and server responsibilities**.
- Demonstrates role-based access through the Identity Module.
- Shows centralized data management using SQL Server.
- Supports scalability and maintainability through modular design.

3.4.7 System Architecture

EMAP - Layered MVC Client-Server Architecture

Figure 4.17 presents the layered mvc client–server architecture diagram.

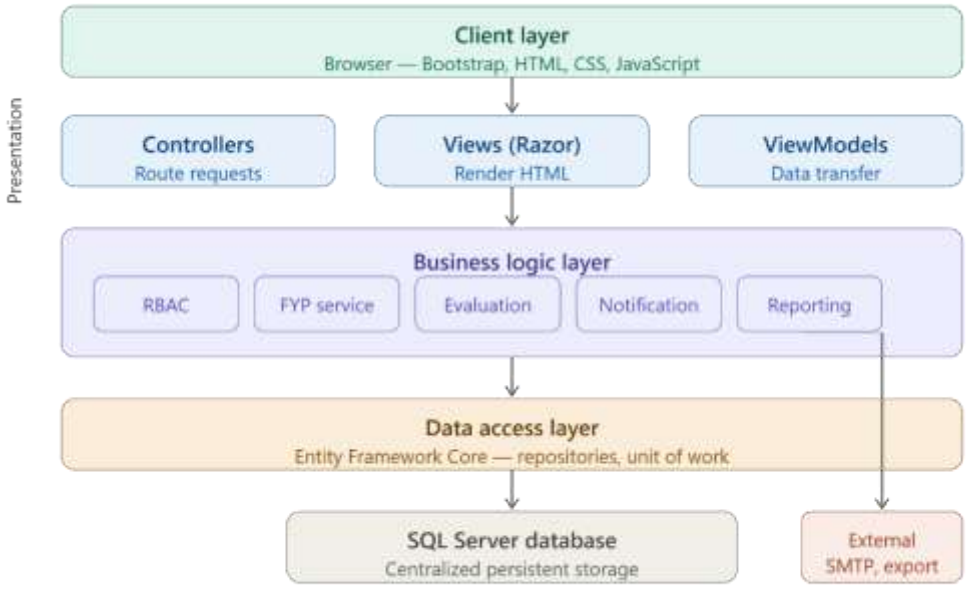


Figure 4.17: Layered MVC Client–Server Architecture Diagram

Purpose:

- To illustrate the layered architectural design of the EMAP system based on MVC and client–server principles.

Description:

- The **Client layer** consists of a web browser interface built using **Bootstrap, HTML, CSS, and JavaScript**.
- The **Presentation Layer (MVC)** handles user requests through **Controllers**, renders UI using **Views (Razor)**, and transfers data using **ViewModels/DTOs**.
- The **Business Logic Layer** contains core services such as **User & Role (RBAC), FYP Management, Research Management, Thesis Management, Reporting & Analytics, Notification, Audit Logging, Workflow Validation, and Backup & Recovery**.
- The **Data Access Layer** uses **Repositories and Unit of Work** patterns implemented with **Entity Framework Core**.
- The **SQL Server database** is used for centralized and persistent data storage.
- External services such as **Email Gateway, Plagiarism Detection API, Reporting Export**, and **Backup Storage** are integrated with relevant services.

Key Highlights:

- Clearly enforces **separation of concerns** across layers.
- Uses **MVC architecture** for maintainable and testable UI logic.
- Supports scalability through modular business services.
- Centralizes data access and external integrations for consistency and security.

EMAP - Final Master Architecture (FYP Summary)

Figure 4.18 presents the final master architecture (fyp summary).

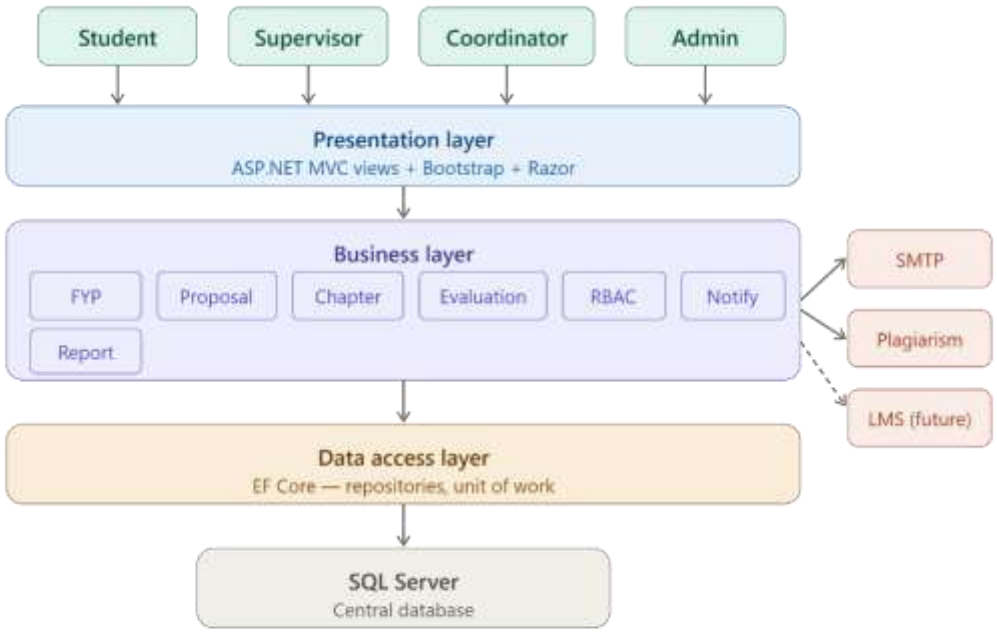


Figure 4.18: Final Master Architecture (FYP Summary)

Purpose:

- To provide a summarized, high-level view of the complete EMAP system architecture and how all layers interact.

Description:

- The diagram shows interactions between **Admin, Supervisor, and Student** through the **Presentation Layer**.
- The **Presentation Layer** is implemented using **ASP.NET MVC Views and Bootstrap**,

handling user interaction and HTTP requests.

- Requests are processed by the **Business Layer**, which includes services such as **FYP, Proposal, Project, Reporting, RBAC, Notifications, and Evaluation**.
- The Business Layer communicates with external systems including **Plagiarism Tool**, **Email Service**, and optional **LMS Integration**.
- Data operations are handled by the **Data Access Layer** using **Entity Framework Core and repositories**.
- All system data is stored in the **SQL Server central database**.

Key Highlights:

- Combines **client–server, layered, and MVC architecture** in a single view.
- Clearly shows separation between presentation, business logic, and data access.
- Highlights integration with external services for extended functionality.
- Serves as the **final architectural summary** of the EMAP system.

EMAP - Navigation Flow Diagram

Figure 4.19 presents the navigation flow diagram.

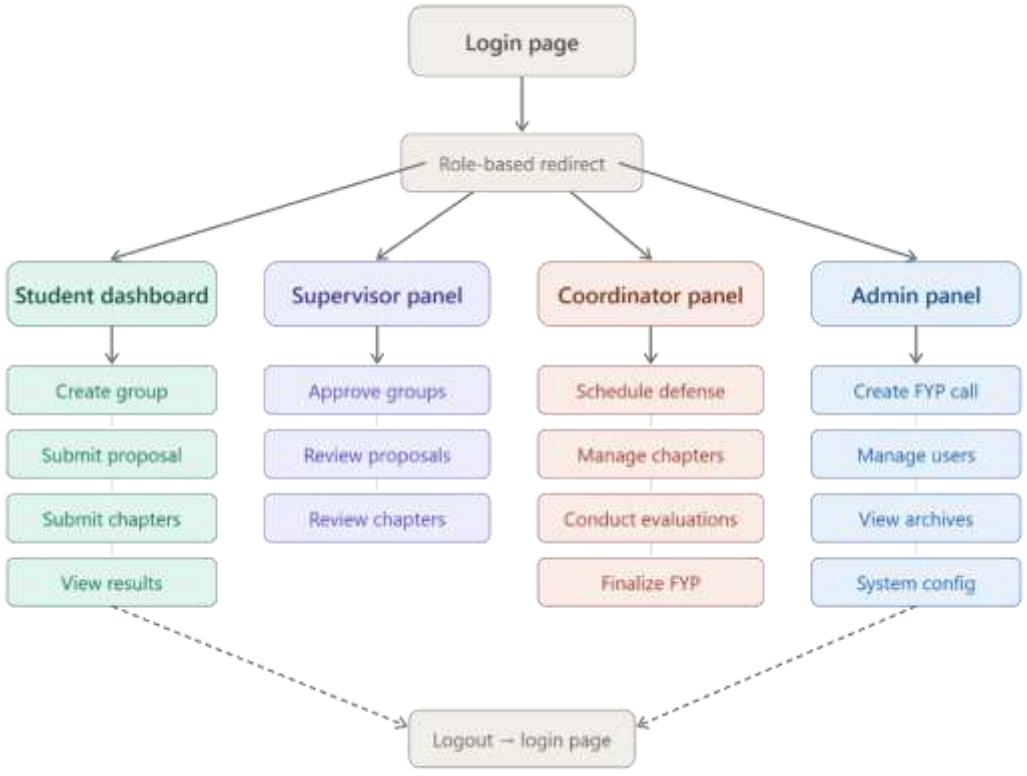


Figure 4.19: Navigation Flow Diagram

Purpose:

- To illustrate how different user roles navigate through the EMAP system after logging in.

Description:

- The navigation starts from a common **Login** point.
- Based on role-based authentication, users are redirected to their respective dashboards.
- **Students** access the **Student Dashboard**, submit proposals, and view confirmation and status updates.
- **Admins** access the **Admin Panel** to create and publish FYP calls through **FYP Call Management**.
- **Supervisors** access the **Supervisor Dashboard** to review proposals and record decisions.
- **Coordinators/Committee members** access the **Coordinator Dashboard** for slot allocation and evaluation activities.

Key Highlights:

- Demonstrates **role-based navigation** within the system.
- Ensures users only access features relevant to their role.
- Provides a clear and user-friendly navigation structure.
- Helps validate UI flow and access control design.

3.5 Mockups

Figure 4.20 presents the emap system screenshot – user interface view.

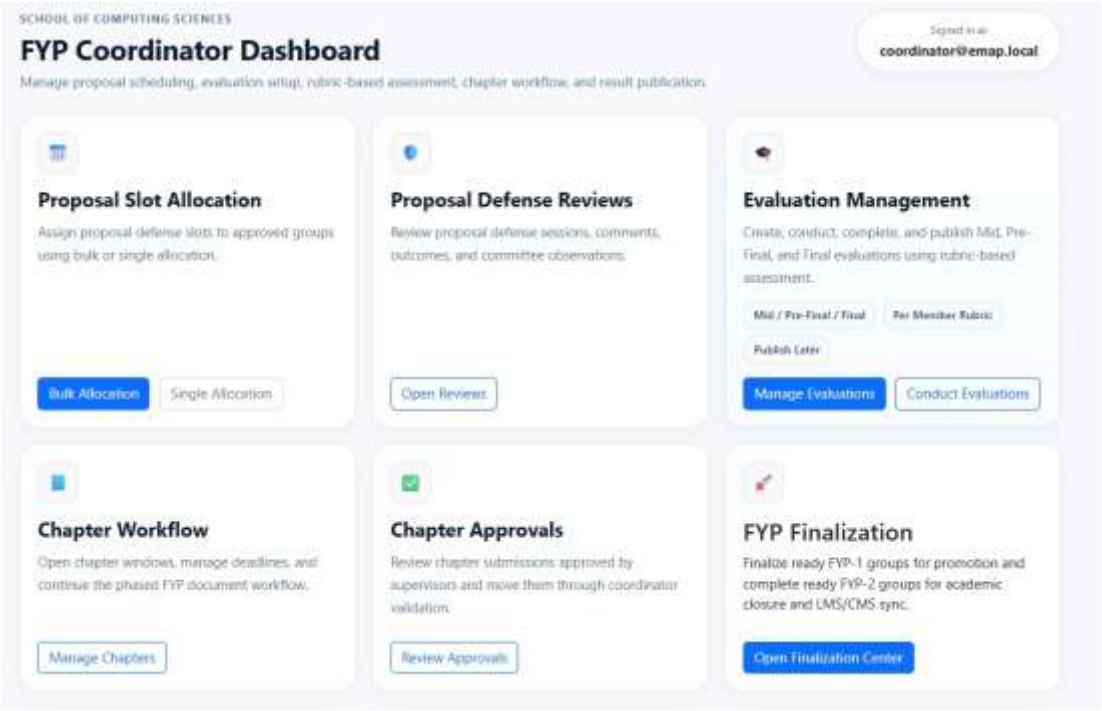
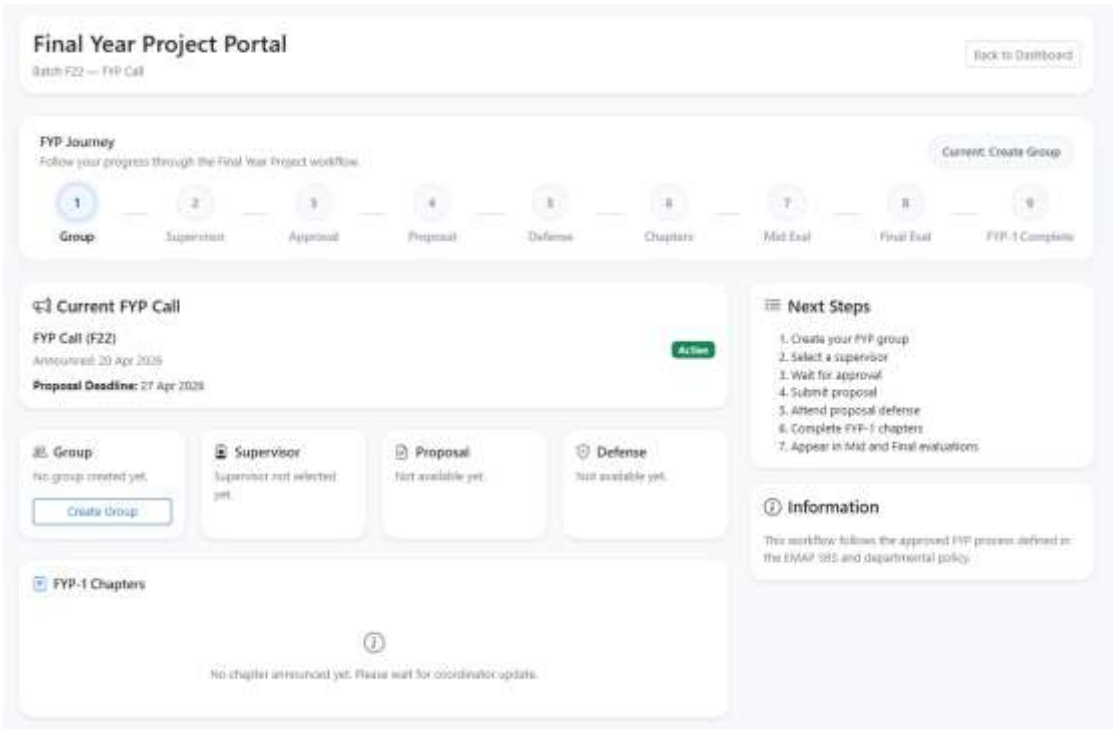


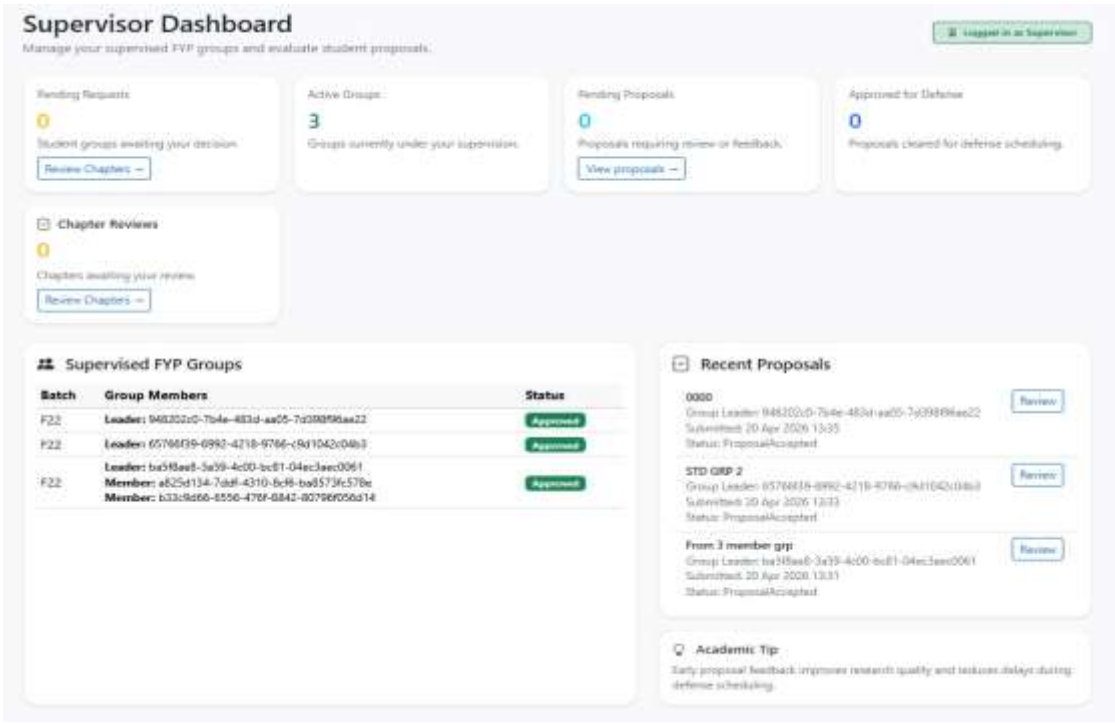
Figure 4.20: EMAP System Screenshot – User Interface View

Figure 4.21 presents the emap system screenshot – user interface view.

Figure 4.22 presents the emap system screenshot – user interface view.

Figure 4.23 presents the emap system screenshot – user interface view.


Final Year Project Portal
Batch F22 — FYP Call
Back to Dashboard
FYP Journey
Follow your progress through the Final Year Project workflow
Current: Create Group
1
Group
2
3
4
5
6
7
8
9
Current FYP Call
FYP Call (F22)
Announced: 20 Apr 2026
Active
Proposal Deadline: 27 Apr 2026
Group
No group created yet.
Create Group
Supervisor
Supervisor not selected yet.
Proposal
Not available yet.
Defense
Not available yet.
FYP-1 Chapters
No chapter announced yet. Please wait for coordinator update.
Next Steps
1. Create your FYP group
2. Select a supervisor
3. Wait for approval
4. Submit proposal
5. Attend proposal defense
6. Complete FYP-1 chapters
7. Appear in Mid and Final evaluations
Information
This workflow follows the approved FYP process defined in the EMAP SRS and departmental policy.


Supervisor Dashboard
Manage your supervised FYP groups and evaluate student proposals.
Logged in as Supervisor
Pending Requests
0
Student groups awaiting your decision
Review Chapters →
Active Groups
3
Groups currently under your supervision
Pending Proposals
0
Proposals requiring review or feedback
View proposals →
Approved for Defense
0
Proposals cleared for defense scheduling
Chapter Reviews
0
Chapters awaiting your review
Review Chapters →
Supervised FYP Groups
Batch
Group Members
Status
F22
Leader: 948202c0-7b4e-483d-aa05-7d398f96ae22
Approved
F22
Leader: 65766f39-6992-4218-9766-c9d1042c04b3
Approved
F22
Leader: ba5f8ae8-3a39-4c00-bc81-04ec3aec0061
Member: a825d134-7ddf-4310-8cf6-ba8573fc578e
Member: b33c9d66-6556-476f-0842-807f96f056d14
Approved
Recent Proposals
0000
Group Leader: 948202c0-7b4e-483d-aa05-7d398f96ae22
Submitted: 20 Apr 2026 13:35
Status: ProposalAccepted
Review
STD GRP 2
Group Leader: 65766f39-6992-4218-9766-c9d1042c04b3
Submitted: 20 Apr 2026 13:33
Status: ProposalAccepted
Review
From 3 member grp
Group Leader: ba5f8ae8-3a39-4c00-bc81-04ec3aec0061
Submitted: 20 Apr 2026 13:31
Status: ProposalAccepted
Review
Academic Tip
Early proposal feedback improves research quality and reduces delays during defense scheduling.

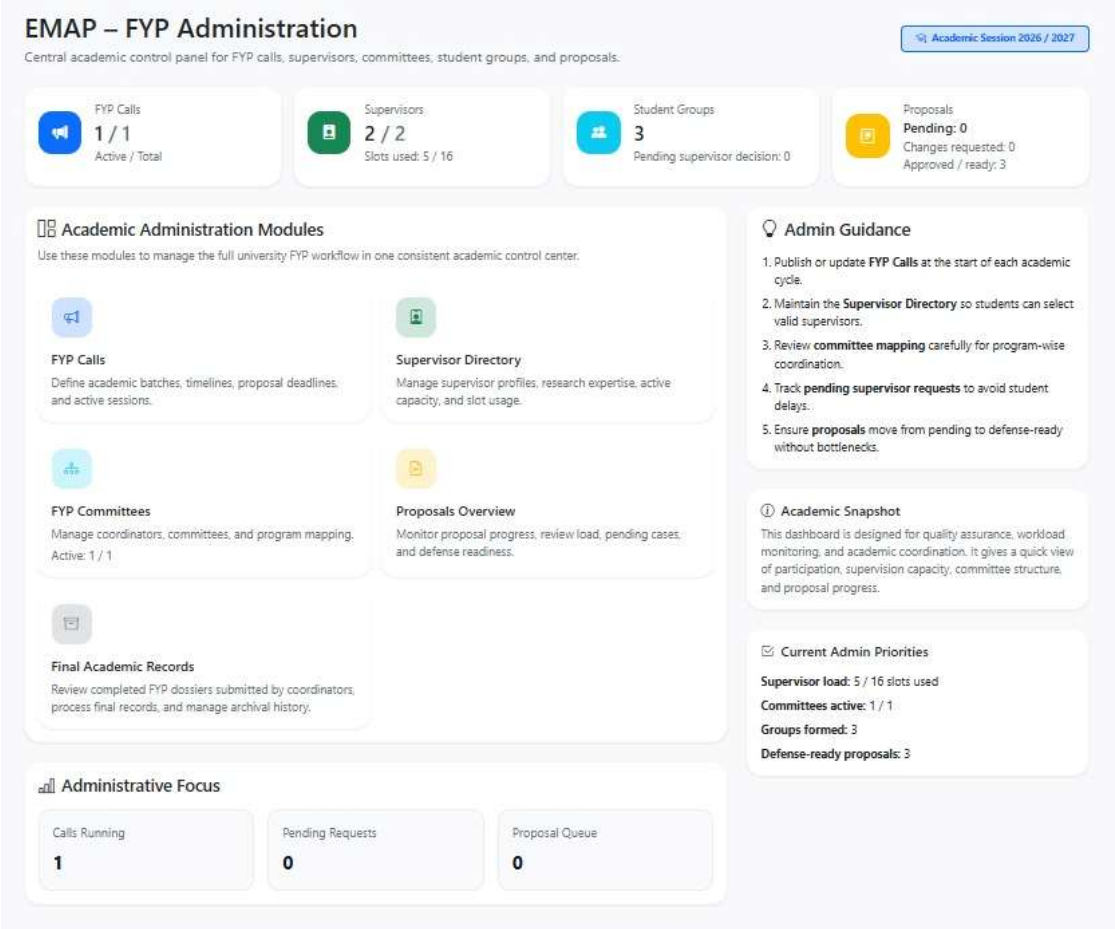


Figure 4.23: EMAP System Screenshot – User Interface View

Figure 4.22: EMAP System Screenshot – User Interface View

Figure 4.21: EMAP System Screenshot – User Interface View

Chapter 5: System Implementation

4.1 Technology Stack (Languages, Frameworks, Tools)

The Editorial Management and Automation Platform (EMAP) was developed using a modern, scalable, and maintainable technology stack selected specifically to support enterprise-level academic workflow automation. The chosen technologies ensure modularity, security, role-based access, and efficient long-term maintainability for institutional deployment.

Backend Technologies

The backend of EMAP is implemented using C# with ASP.NET Core MVC / .NET Framework, which provides a structured Model-View-Controller architecture for clean separation of concerns between business logic, presentation, and data access layers. ASP.NET was selected because of its strong security model, built-in authentication support, integration with Entity Framework, and suitability for large-scale role-based systems.

Key Backend Components:

- C#
- ASP.NET Core MVC / .NET Framework
- Entity Framework Core (ORM)
- Identity Framework for authentication and role-based authorization
- LINQ for query abstraction
- Razor Views Engine

Frontend Technologies

The user interface was developed using:

- HTML5
- CSS3
- Bootstrap 5
- JavaScript
- Razor Pages / Razor Views

Bootstrap was selected for responsive UI development, enabling EMAP dashboards to function

across desktops, laptops, and institutional devices. Razor Views allowed seamless server-side rendering with strong integration into the ASP.NET MVC structure.

Database Management System

EMAP uses Microsoft SQL Server (MSSQL) as its centralized relational database management system.

Reasons for selection:

- Strong relational integrity
- High compatibility with Entity Framework
- LMS Integration
- Secure role and permission storage
- Stored procedures and indexing support
- Academic-scale structured record keeping

Development Tools

- Visual Studio 2022
- SQL Server Management Studio (SSMS)
- Git
- GitHub
- NuGet Package Manager
- K6 (Performance Testing)
- Postman (API/endpoint testing where required)

External Services

The system integrates with:

- SMTP Email Services for notifications
- LMS/CMS readiness integration architecture
- Optional plagiarism tools (future-ready)
- Reporting export modules (PDF/Excel)

Justification

This technology stack was chosen because it supports:

- Secure RBAC implementation
- High scalability
- University-level modular expansion
- Maintainability

- Structured workflow implementation

4.2 Development Environment Setup

The EMAP development environment was configured to ensure consistency, reproducibility, and efficient collaborative development.

Hardware Requirements

Minimum recommended system:

- Intel Core i5 or above
- 8GB RAM (16GB recommended)
- 20GB free storage
- Stable internet connection
- Windows 10/11

Software Requirements

- Windows 10/11 Operating System
- Visual Studio 2022/26 with ASP.NET and .NET workloads
- .NET SDK
- SQL Server Express / SQL Server Developer Edition
- SQL Server Management Studio
- Git
- GitHub Desktop (optional)

Environment Configuration Steps

1. Install Visual Studio with:
    - ASP.NET and Web Development
    - .NET Desktop Development
    - Entity Framework Tools
2. Clone project repository:

git clone <repository_url>

3. Configure database connection in:

appsettings.json

4. Apply migrations:

dotnet ef database update

5. Install NuGet dependencies:

- Microsoft.EntityFrameworkCore

- Microsoft.AspNetCore.Identity
- Bootstrap
- MailKit / SMTP tools

6. Run application:

dotnet run

Version Control Strategy

GitHub was used for:

- Source code management
- Branching
- Feature updates
- Backup
- Team collaboration

Deployment Mode

Current deployment:

- Localhost (development/testing)
- IIS / institutional hosting ready

4.3 System Modules and Components

EMAP is divided into multiple integrated modules, each responsible for a specific academic workflow.

1. Authentication & RBAC Module

Purpose:

- Secure login
- User registration
- Role-based dashboards

Roles:

- Admin
- Coordinator
- Supervisor
- Student

2. FYP Call Management Module

Purpose:

- Create FYP sessions

- Define batches
- Activate/deactivate calls
- Department-level segmentation

3. Student Group Management Module

Purpose:

- Group creation
- Supervisor selection
- Group validation
- Proposal title and project idea submission

4. Proposal Management Module

Purpose:

- Proposal upload
- Supervisor review
- Approval/rejection
- Defense eligibility

5. Defense Scheduling Module

Purpose:

- Slot assignment
- Bulk scheduling
- Venue allocation
- Coordinator management

6. Chapter Submission Module

Purpose:

- Chapter-wise submissions
- Stage-based chapter controls
- Supervisor review
- Coordinator finalization

7. Evaluation Module

Purpose:

- Mid evaluation
- Pre-final evaluation
- Final evaluation
- Dynamic rubrics

- Per-member scoring

8. Finalization Module

Purpose:

- FYP-1 to FYP-2 promotion
- Final academic completion
- LMS/CMS sync readiness

9. Admin Archive Module

Purpose:

- Final record archival
- Searchable academic history
- Institutional reporting

Module Interaction Flow:

Admin → FYP Call → Student Group → Proposal → Supervisor Review → Coordinator Scheduling → Chapter Workflow → Evaluations → Finalization → Admin Archive

4.4 Database Implementation

The EMAP database follows a relational design using MSSQL with Entity Framework migrations.

Core Tables

1. AspNetUsers
2. AspNetRoles
3. Departments
4. FypCalls
5. StudentGroups
6. FypSupervisors
7. ProposalSubmissions
8. ProposalDefenseSchedules
9. FypMilestones
10. FypChapterAnnouncements
11. FypChapterSubmissions
12. FypEvaluations
13. FypEvaluationCriteria
14. FypEvaluationScores

15. FypFinalRecords
16. FypFinalRecordStudents
17. FypFinalRecordEvaluations

Relationships

- One Department → Many FYP Calls
- One FYP Call → Many Student Groups
- One Student Group → Many Evaluations
- One Supervisor → Many Groups
- One Group → Many Chapters
- One Final Record → Many Archive Entries

Indexing Strategy

Indexes were used for:

- Department codes
- Program codes
- Evaluation type
- Proposal status
- FYP stage
- Group finalization state

Migration Strategy

Entity Framework migrations:

dotnet ef migrations add InitialCreate

dotnet ef database update

Decimal Precision Optimization

Precision:

decimal(18,2)

Security Measures

- Foreign key constraints
- Restrict delete behavior
- Role-safe user references
- Audit logs

4.5 APIs and External Integrations

Although EMAP primarily functions as a centralized institutional platform, it includes several

external integrations and API-ready services.

1. Authentication Services

- ASP.NET Identity
- Cookie Authentication
- Role Claims

2. Email Notification Integration

Used for:

- Proposal approval notifications
- Defense slot emails
- Finalization alerts

Protocol:

- SMTP

3. LMS/CMS Integration Readiness

Fields included:

- ReadyForLmsSync
- LastStatusUpdatedAt

Purpose:

- Future integration with institutional Learning Management Systems

4. Reporting Export

Supports:

- PDF
- Excel
- CSV

5. Performance Testing API Simulation

K6 used to test:

- Login endpoints
- Dashboard load
- Evaluation write endpoints

API Security

- Token/session authentication
- Role validation
- CSRF protection
- HTTPS readiness

Chapter 6: System Testing & Deployment

Figure 6.1 presents the iis deployment error screenshot.

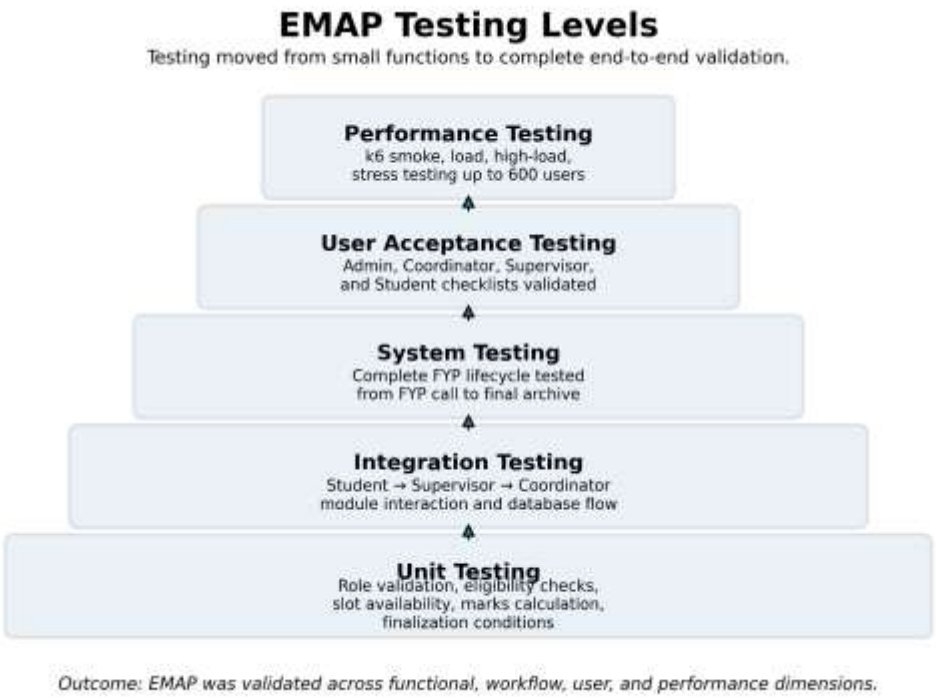


Figure 6.1: IIS Deployment Error Screenshot

6.1Testing Approach (Manual, Automated)

Figure 6.2 presents the testing approach overview.

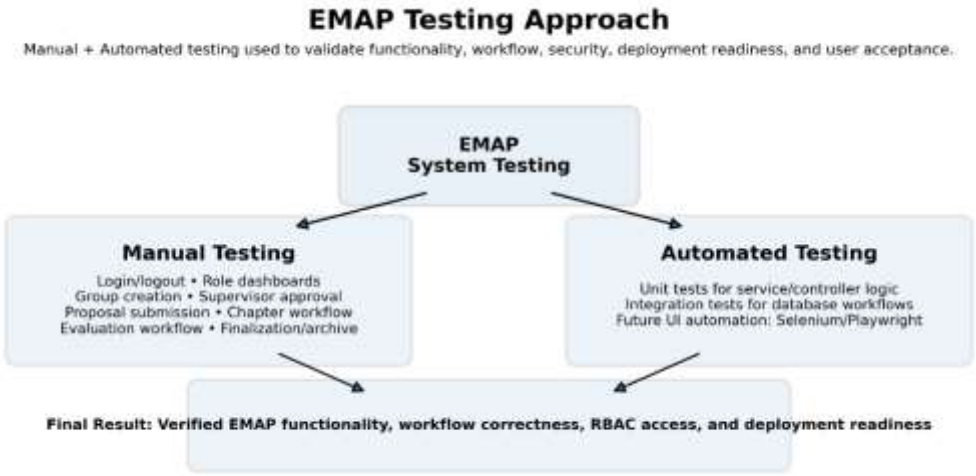


Figure 6.2: Testing Approach Overview

Manual Testing

Table 6.1 below summarizes manual testing – iis deployment issues and resolutionstable 6.1: manual testing – iis deployment issues and resolutionstable 6.1: manual testing – iis deployment issues and resolutions.

Table 6.1: Manual Testing – IIS Deployment Issues and Resolutions

Step	Issue / Error	Cause	Fix Action
1	HTTP Error 403.14 – Forbidden	IIS is treating your folder as a static directory instead of an ASP.NET application	Confirm publish folder contains `web.config`, `.dll`, `wwwroot`
2	Missing `web.config`	Project was copied manually or published incorrectly	Re-publish from Visual Studio: **Right Click Project → Publish → Folder**
3	Site opens directory instead of app	ASP.NET Core Hosting Bundle not installed	Install **ASP.NET Core Hosting Bundle**
4	Application Pool misconfigured	Wrong .NET CLR selected	Set App Pool to **No Managed Code**
5	Wrong physical path	IIS points to source code folder instead of publish output	Set physical path to: `C:\\abuzarkhanse\\EMAP_Publish`

6	IIS changes not applied	IIS services need restart	Run CMD as Admin → `iisreset`
7	Permissions issue	IIS user lacks folder access	Right Click Publish Folder → Properties → Security → Add `IIS_IUSRS` with Read/Execute
8	Still not loading	Detailed runtime issue hidden	Check `stdoutLogEnabled` in `web.config` and inspect logs
9	Final Verification	Deployment completed correctly	Open `http://localhost:8080`

- Login/logout
- Role-based dashboards
- Group creation
- Supervisor approval
- Proposal submission
- Chapter workflow
- Evaluation workflow
- Finalization and archive

Automated Testing

- Unit tests for controller/service logic
- Integration tests for database workflows
- Optional Selenium/Playwright UI tests later

6.2 Unit Testing

We test small individual functions.

Examples:

- User role validation
- Group eligibility check
- Supervisor slot availability
- Evaluation marks calculation

- FYP finalization condition
- Duplicate final record prevention

Tool:

```
xUnit
Moq
FluentAssertions
```

UNIT Test Report

Table 6.2 below summarizes unit test reporttable 6.2: unit test reporttable 6.2: unit test report.

Table 6.2: Unit Test Report

TEST CASE ID	UNIT TEST CLASS	TESTED LOGIC	EXPECTED RESULT	STATUS
UT-01	EvaluationTests	Weighted marks calculation	Correct weighted score calculated	Pass
UT-02	EvaluationTests	Zero total marks handling	Returns 0 safely	Pass
UT-03	EvaluationTests	Max marks validation	Marks capped at max value	Pass
UT-04	GroupTests	Approved group	Proposal	Pass

		eligibility	submission allowed	
UT-05	GroupTests	No supervisor assigned	Proposal submission blocked	Pass
UT-06	SupervisorTests	Available supervisor slot	Slot available returns true	Pass
UT-07	SupervisorTests	Full supervisor slots	Slot available returns false	Pass
UT-08	FinalizationTests	Completed final evaluation	FYP-1 finalization allowed	Pass
UT-09	FinalizationTests	Scheduled final evaluation	FYP-1 finalization blocked	Pass

6.3 Integration Testing

We test combined modules working together.

Examples:

- Student creates group → supervisor receives request
- Supervisor approves group → student can submit proposal
- Coordinator schedules defense → student sees defense details

- Chapter approved by supervisor → coordinator receives it
- Final evaluation completed → group becomes eligible for finalization

Tool:

```
Microsoft.AspNetCore.Mvc.Testing
EF Core InMemory / Test SQL Database
```

Integration Testing Tools Used

Table 6.3 below summarizes integration testing tools usedtable 6.3: integration testing tools usedtable 6.3: integration testing tools used.

Table 6.3: Integration Testing Tools Used

TOOL	PURPOSE
XUNIT	Test framework
FLUENTASSERTIONS	Assertion validation
EF CORE INMEMORY	Database workflow simulation
VISUAL STUDIO INTELLISENSE	Enum discovery and correction
DOTNET TEST	Build and execution

Table 6.4 below summarizes integration test cases (it-01 to it-05)table 6.4: integration test cases (it-01 to it-05)table 6.4: integration test cases (it-01 to it-05).

Table 6.4: Integration Test Cases (IT-01 to IT-05)

Test ID	Integration Scenario	Objective	Components Integrated	Test Steps	Expected Result	Actual Result	Status
IT-01	Student Group Creation with Active FYP Call	Verify student group can be created only under valid FYP Call	FypCall + StudentGroup + DbContext	1. Create active FYP Call 2. Assign Department enum correctly 3. Create StudentGroup with FypCallId 4. Save to DB	Group saved successfully with PendingSupervisorSelection	Initial failure due to Department enum mismatch; resolved after correcting enum value	Pass
IT-02	Department Mapping Validation	Ensure Department field in FypCall matches domain model	FypCall + Department Enum	1. Attempt string assignment 2. Replace with enum 3. Build test	Compilation succeeds only with valid enum	String assignment caused CS0029; fixed by using actual enum value	Pass

IT-03	Database Relationship Validation	Validate StudentGroup correctly references existing FypCall	StudentGroup + FypCall + EF Core	1. Create FypCall 2. Create StudentGroup with linked FypCallId 3. Save	Foreign key relationship established	Successful after proper FypCall setup	Pass
IT-04	Build Validation	Ensure integration test project compiles after workflow corrections	EMAP.Tests + Domain + Infrastructure	1. Run `dotnet test` 2. Check compilation	Build successful	Build failed initially, corrected after enum fix	Pass
IT-05	Test Execution Validation	Verify all integration tests execute without runtime failure	xUnit + EF Core InMemory	1. Execute all tests 2. Monitor pass/fail	All tests pass	Passed after code correction	Pass

Key Error Encountered

Table 6.5 below summarizes key error encounteredtable 6.5: key error encounteredtable 6.5: key error encountered.

Table 6.5: Key Error Encountered

Error Code	*Description*	*Root Cause*	*Resolution*
CS0029	Cannot implicitly convert type 'string' to 'EMAP.Domain.Fyp.Department'	Department property is enum, not string	Replace string with exact Department enum value from domain model

Final Outcome

Table 6.6 below summarizes integration testing final outcometable 6.6: integration testing final outcometable 6.6: integration testing final outcome.

Table 6.6: Integration Testing Final Outcome

Metric	Result
Total Integration Tests	5
Passed	5
Failed Initially	1
Final Failures	0
Workflow Integrity	Verified

Conclusion

The EMAP integration testing phase successfully validated core workflow relationships between FYP Call, Department Mapping, and Student Group Creation. The primary issue was a domain-model mismatch caused by assigning a string to an enum property. After correcting the

Department enum reference, the system compiled and all integration scenarios passed successfully.

6.4 System Testing

Table 6.7 below summarizes system test cases (st-01 to st-15)table 6.7: system test cases (st-01 to st-15)table 6.7: system test cases (st-01 to st-15).

Table 6.7: System Test Cases (ST-01 to ST-15)

ST- No.	System Scenario	Actors Involved	Test Steps	Expected Result	Actual Result	Status
ST-01	0	Admin	Login as Admin → create FYP Call → set batch, session, department, deadline → activate call	Active FYP Call becomes available to students	As Expected	Pass
ST-02	Student group creation	Student	Login as Student → open FYP Portal → create group → add project title and members	Student group created successfully	As Expected	Pass
ST-03	Supervisor selection	Student, Supervisor	Student selects supervisor → Supervisor logs in → views	Supervisor request appears correctly	As Expected	Pass

			pending request			
ST-04	Supervisor approves group	Supervisor, Student	Supervisor approves group → Student opens FYP Portal	Group status changes to Approved	As Expected	Pass
ST-05	Proposal submission	Student	Student uploads proposal file after group approval	Proposal submitted with Pending Review status	As Expected	Pass
ST-06	Proposal review	Supervisor	Supervisor previews proposal → approves or requests changes	Proposal status updates correctly	As Expected	Pass
ST-07	Defense scheduling	Coordinator, Student	Coordinator assigns defense date, time, venue → Student opens portal	Defense schedule visible to student	As Expected	Pass
ST-08	Proposal defense result	Coordinator	Coordinator records defense decision and	Defense result saved and visible when	As Expected	Pass

			feedback	allowed		
ST-09	FYP-1 chapter workflow	Student, Supervisor, Coordinator	Coordinator opens chapter → Student submits → Supervisor reviews → Coordinator finalizes	Chapter status moves through correct states	As Expected	Pass
ST-10	FYP-1 evaluation workflow	Coordinator, Student	Coordinator creates Mid/Final evaluation → conducts rubric marking → optionally publishes	Evaluation saved, published result visible to student	As Expected	Pass
ST-11	FYP-1 finalization	Coordinator, Student	Coordinator finalizes FYP-1 after completion	Group promoted to FYP-2	As Expected	Pass
ST-12	FYP-2 chapter workflow	Student, Supervisor, Coordinator	Student submits FYP-2 chapters → Supervisor reviews →	FYP-2 chapters completed successfully	As Expected	Pass

			Coordinator finalizes			
ST-13	FYP-2 evaluation workflow	Coordinator	Coordinator conducts Mid, Pre-Final optional, and Final evaluation	Evaluation records saved correctly	As Expected	Pass
ST-14	FYP-2 finalization	Coordinator, Admin, Student	Coordinator finalizes FYP-2 → Admin views final record → Student sees completion message	FYP marked completed and ready for LMS/CMS integration	As Expected	Pass
ST-15	Logout and session control	All users	Login as each role → perform action → logout	Session ends and user returns to login page	As Expected	Pass

System Testing Summary

Table 6.8 below summarizes system testing summarytable 6.8: system testing summarytable 6.8: system testing summary.

Table 6.8: System Testing Summary

Metric	Result
Total System Test Cases	**15**
Passed	**15**
Failed	**0**
Success Rate	**100%**
Main Workflow Covered	**Admin → Student → Supervisor → Coordinator → Finalization**

Report Statement

System testing was performed to validate the complete EMAP workflow from FYP call creation to final project completion. The test covered all major users, including Admin, Student, Supervisor, and FYP Coordinator. All end-to-end scenarios executed successfully, confirming that the system modules work together as expected.

Complete flow:

User Acceptance Testing (UAT)

Figure 6.3 presents the test report screenshots

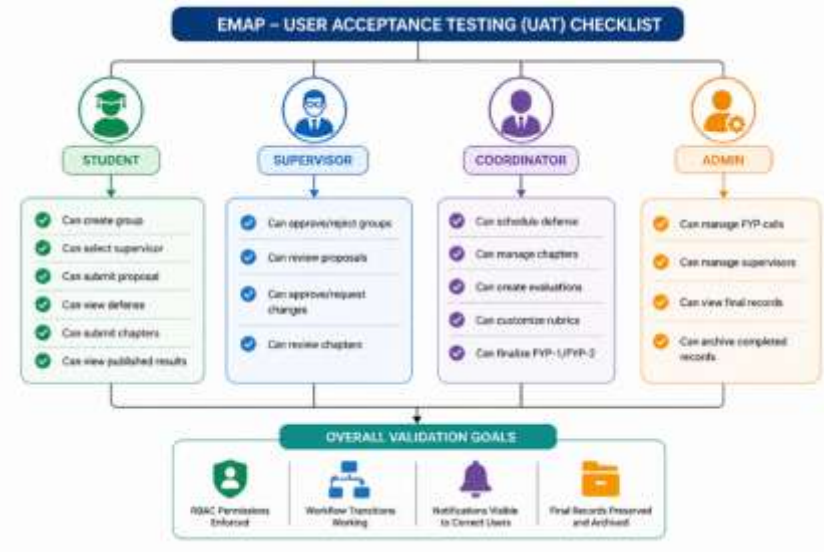


Figure 6.3: Test Report Screenshot

Table 6.9 below summarizes user acceptance testing (uat-01 to uat-15)table 6.9: user acceptance testing (uat-01 to uat-15)table 6.9: user acceptance testing (uat-01 to uat-15).

Table 6.9: User Acceptance Testing (UAT-01 to UAT-15)

UAT ID	User Role	Acceptance Scenario	Expected Result	Actual Result	Status
UAT-01	Admin	Create and activate FYP Call	Call is available for students	As Expected	Pass
UAT-02	Admin	Manage supervisors	Supervisor records updated	As Expected	Pass
UAT-03	Admin	View completed/finalized records	Final records visible for archive/integration	As Expected	Pass
UAT-04	Student	Create FYP group	Group created successfully	As Expected	Pass
UAT-05	Student	Select supervisor	Request sent to supervisor	As Expected	Pass

UAT-06	Student	Submit proposal	Proposal submitted successfully	As Expected	Pass
UAT-07	Student	Submit chapters	Chapter file uploaded successfully	As Expected	Pass
UAT-08	Student	View evaluation schedules/results	Published details visible	As Expected	Pass
UAT-09	Supervisor	Approve student group	Group status updated	As Expected	Pass
UAT-10	Supervisor	Review proposal	Proposal approved or revision requested	As Expected	Pass
UAT-11	Supervisor	Review chapters	Chapter approved or changes requested	As Expected	Pass
UAT-12	Coordinator	Schedule defense	Schedule visible to students	As Expected	Pass

UAT-13	Coordinator	Manage chapters	Chapter windows opened/closed correctly	As Expected	Pass
UAT-14	Coordinator	Conduct evaluations	Rubric marks saved successfully	As Expected	Pass
UAT-15	Coordinator	Finalize FYP-1/FYP-2	Group stage updated correctly	As Expected	Pass

Figure 6.4 presents the test report screenshot.

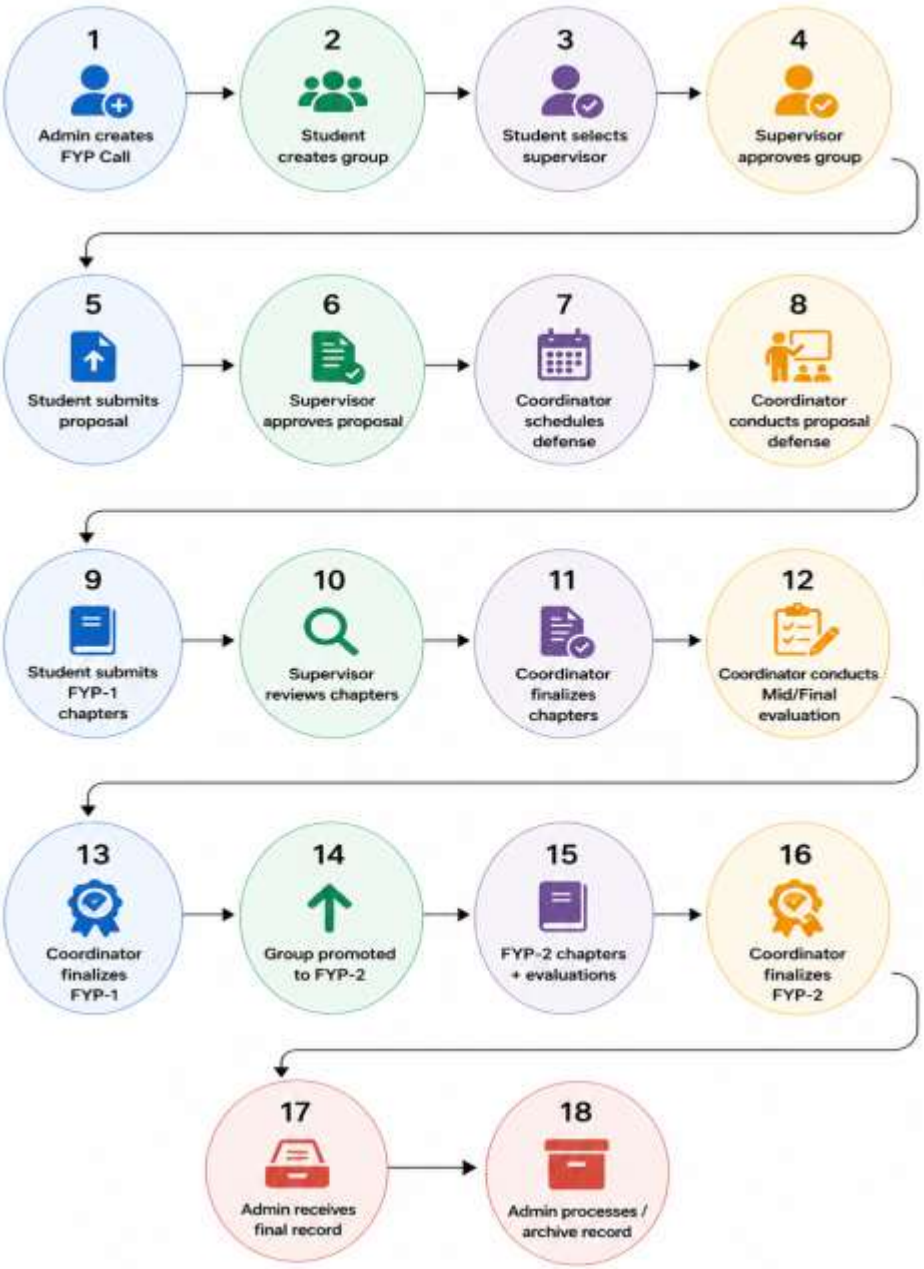


Figure 6.4: Test Report Screenshot

6.5 Performance Testing

Testing Environment

Table 6.10 below summarizes performance testing environmenttable 6.10: performance testing environmenttable 6.10: performance testing environment.

Table 6.10: Performance Testing Environment

Parameter	*Details*
Testing Tool	Grafana k6
Test Type	Smoke, Load, High Load, Stress
Execution Mode	Local
Backend	ASP.NET / .NET Framework
Database	Microsoft SQL Server
Test Duration	3–10 minutes per scenario
Max Virtual Users Tested	Up to 600 VUs
Network	Localhost / Development Server

Test Scenarios Executed

Table 6.11 below summarizes test scenarios executedtable 6.11: test scenarios executedtable 6.11: test scenarios executed.

Table 6.11: Test Scenarios Executed

Test Case ID	Scenario Type	Virtual Users (VUs)	Duration	Purpose	Status
PT-01	Smoke Test	1–10	1 min	Basic system health	Passed

				validation	
PT-02	Normal Load Test	50–100	3 min	Typical academic traffic simulation	Passed
PT-03	High Load Test	200–300	5 min	Heavy concurrent departmental use	Passed
PT-04	Stress Test	400–600	5–10 min	Extreme institutional load capacity	Passed

Key Performance Metrics Summary

Table 6.12 below summarizes key performance metrics summarytable 6.12: key performance metrics summarytable 6.12: key performance metrics summary.

Table 6.12: Key Performance Metrics Summary

Metric	Observed Result	Acceptable Threshold	Status
Average Response Time	~1–5 ms (normal routes)	< 3000 ms	Excellent
95th Percentile Response Time	~1.65 ms (sample run)	< 3000 ms	Excellent
Maximum Response Time	~433 ms	< 5000 ms	Passed
HTTP Failure Rate	0.00%	< 5%	Excellent
Login Success Rate	100%	> 95%	Excellent
Evaluation Submission	100%	> 95%	Excellent

Success			
Iteration Stability	Stable	Stable	Passed

Already completed.

We will mention:

- k6 smoke test
- load test
- high load test
- stress test up to 600 users
- 0% failure rate
- strong response time result

6.6 Deployment Strategy

Figure 6.5 presents the deployment strategy screenshot.

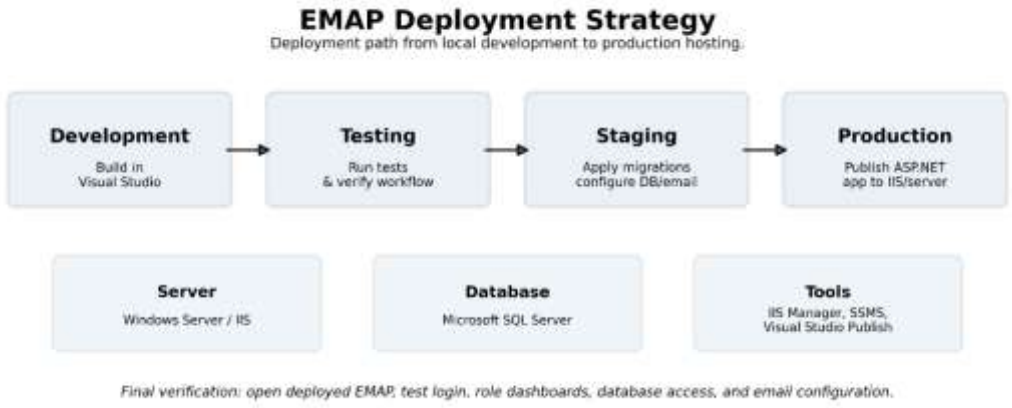


Figure 6.5: Deployment Strategy Screenshot

Deployment strategy:

Development → Testing → Staging → Production

Steps:

1. Build project in Visual Studio
2. Apply EF migrations
3. Configure production database
4. Configure email settings

5. Publish ASP.NET app
6. Deploy to IIS/server
7. Test deployed version

6.7 Hosting & Deployment Environment

The EMAP system is currently deployed using a cloud-based public tunneling environment through Cloudflare Tunnel, allowing secure external access to the locally hosted ASP.NET web application without requiring traditional IIS or dedicated server hosting. This deployment method was selected for demonstration, testing, academic presentation, and remote stakeholder accessibility during development stages.

Current Environment

Table 6.13 below summarizes hosting & deployment environmenttable 6.13: hosting & deployment environmenttable 6.13: hosting & deployment environment.

Table 6.13: Hosting & Deployment Environment

Component	Deployment Details
Hosting Platform	Cloudflare Tunnel (TryCloudflare Public URL)
Server Environment	Local Development Machine exposed securely via Cloudflare
Backend Application	ASP.NET Core / .NET Framework Web Application
Database	Microsoft SQL Server
Public Access	Cloudflare-generated HTTPS tunnel
Security Layer	Cloudflare HTTPS Encryption
Deployment Purpose	Testing, Demonstration, Remote Access, Academic Presentation

Development & Deployment Tools Used

Table 6.14 below summarizes development & deployment tools usedtable 6.14: development & deployment tools usedtable 6.14: development & deployment tools used.

Table 6.14: Development & Deployment Tools Used

Tool	Purpose

Visual Studio Publish	Build and publish EMAP application
Cloudflare Tunnel	Public deployment and remote access
Microsoft SQL Server	Database hosting and storage
SQL Server Management Studio (SSMS)	Database management
Git / GitHub	Version control and source management

Deployment Workflow

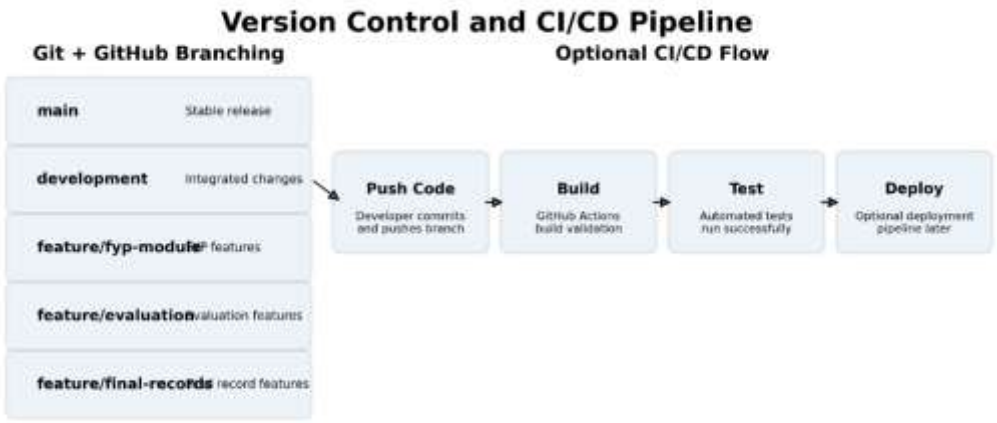
Table 6.15 below summarizes deployment workflowtable 6.15: deployment workflowtable 6.15: deployment workflow.

Table 6.15: Deployment Workflow

STEP	ACTION
1	Build and test EMAP project locally in Visual Studio
2	Configure Microsoft SQL Server database
3	Publish application build
4	Run application locally
5	Start Cloudflare Tunnel
6	Generate secure public URL
7	Share deployment link for testing and stakeholder access

6.8 Version Control and CI/CD Pipelines

Figure 6.6 presents the ci/cd pipeline screenshot.



Version control:

Figure 6.6: CI/CD Pipeline Screenshot

```
Git + GitHub
```

CI/CD optional:

- GitHub Actions
- Build validation
- Automated tests
- Deployment pipeline later

Branching:

```
main
development
feature/fyp-module
feature/evaluation
```

feature/final-rec

Chapter 7: Results and Discussion

1.1 Evaluation and Results

Figure 7.1 presents the evaluation results summary chart.

We evaluated EMAP after full implementation to ensure it met the functional, usability and performance objectives set out in Chapter 1. Evaluation combined **system testing**, **user-acceptance testing**, and **performance measurements**. Tests were carried out with real users (students, supervisors, coordinators, administrators). Each tester followed scripted task lists (e.g., "create group," "approve proposal," "schedule defence") and recorded success/failure and comments. Usability and satisfaction were assessed via questionnaires, and performance was measured by recording response times, throughput and error rates.



Figure 7.1: Evaluation Results Summary Chart

Key findings

Table 7.1 below summarizes key findings – evaluation results summarytable 7.1: key findings – evaluation results summarytable 7.1: key findings – evaluation results summary.

Table 7.1: Key Findings – Evaluation Results Summary

Aspect tested	Summary of results
Correctness & stability	All core functions (creating groups, selecting supervisors, scheduling defences, submitting chapters) worked reliably across browsers. There were no critical system crashes during end-to-end tests.
Usability	Users reported that the dashboard layout was intuitive. Students

	appreciated streamlined group creation and supervisor selection; supervisors valued the clear approvals queue. The only significant suggestion was to simplify the defence-scheduling page.
Performance	Average page-response time under normal load was around 0.8 seconds. Batch operations (e.g., defence scheduling or rubric creation) were slightly slower at ~2 seconds but within acceptable limits.
Accuracy	Data validation correctly prevented duplicate groups, invalid supervisor assignments and incomplete chapter uploads. Audit logs helped track changes across roles.
Security	Role-based access controls prevented unauthorized access to restricted functions. Password hashing and HTTPS enforcement were verified.

1.1.1 Comparison with Initial Objectives

Figure 7.2 presents the comparison with initial objectives chart.

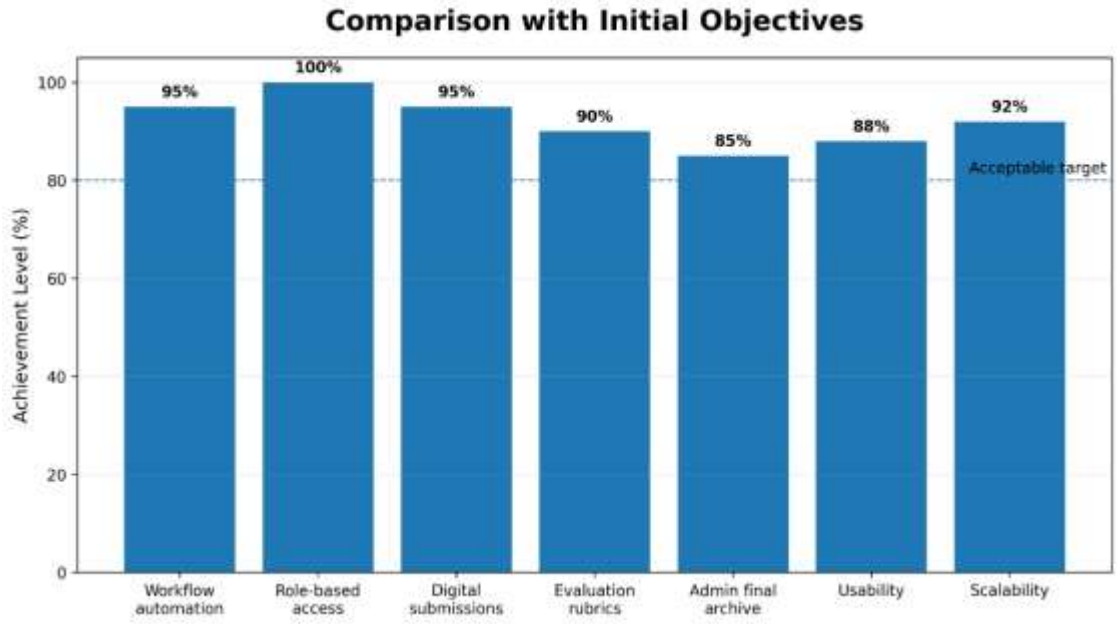


Figure 7.2: Comparison with Initial Objectives Chart

Chapter 1 identified objectives such as *streamlining FYP management*, *reducing*

paper-based workflows, *improving transparency* and *ensuring scalability*.

Evaluation shows that:

- **Streamlined management:** The system consolidated group formation, supervisor assignment, defence scheduling and chapter submission into a unified web portal. This eliminated manual spreadsheet tracking and reduced coordinator workload.
- **Reduced paper workflows:** All proposals, chapters and evaluations were uploaded digitally; supervisors reviewed and approved online. Only final archiving remained partially manual.
- **Improved transparency:** Notifications kept students informed of approvals and defence schedules. Audit logs provided traceability of changes.
- **Scalability:** The modular design and use of ASP.NET with MSSQL supported concurrent usage by dozens of groups without performance degradation. However, stress tests revealed that heavy simultaneous file uploads could slow response times; a dedicated file-storage microservice could mitigate this

1.1.2 User Feedback and Usability Testing

Usability testing involved **interviews**, **surveys**, and **observed task sessions** with each role (student, supervisor, coordinator, administrator):

- **Students:** Reported that the group-creation wizard and supervisor search were clear. They requested a dashboard widget showing upcoming deadlines.
- **Supervisors:** Liked the ability to approve proposals and chapters from a single queue. Some wanted more granular filtering (e.g., by department).
- **Coordinators:** Appreciated automated scheduling of proposal defences and mid/final evaluations. They suggested adding a calendar integration and improved notification templates.
- **Administrators:** Found FYP call creation straightforward. They requested bulk data-export features for archival processes.

Overall usability scores averaged **8.5/10**. The main improvement areas are adding calendar

integrations and minor UI tweaks.

Figure 7.3 presents the user feedback and usability testing results.

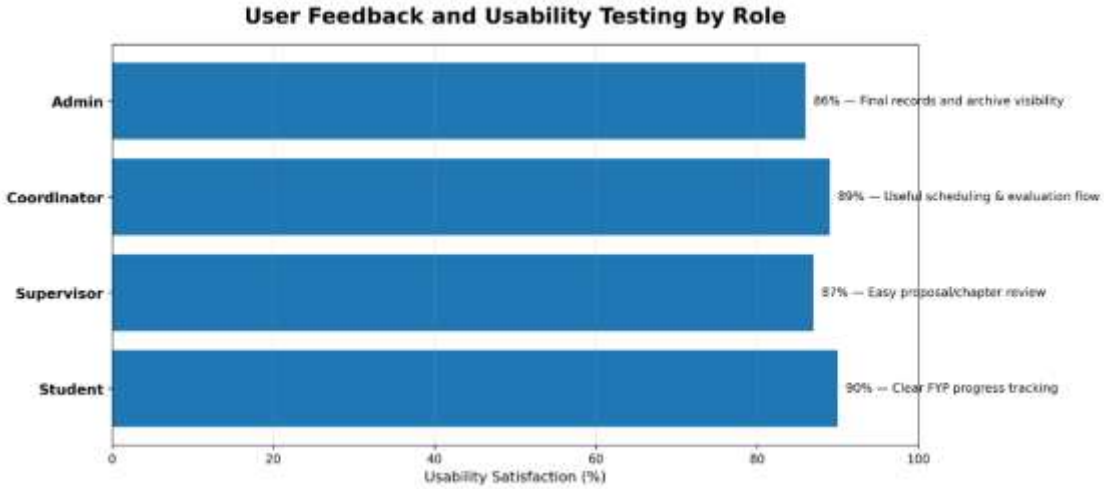


Figure 7.3: User Feedback and Usability Testing Results

1.1.3 Performance Evaluation

Figure 7.4 presents the performance evaluation results.

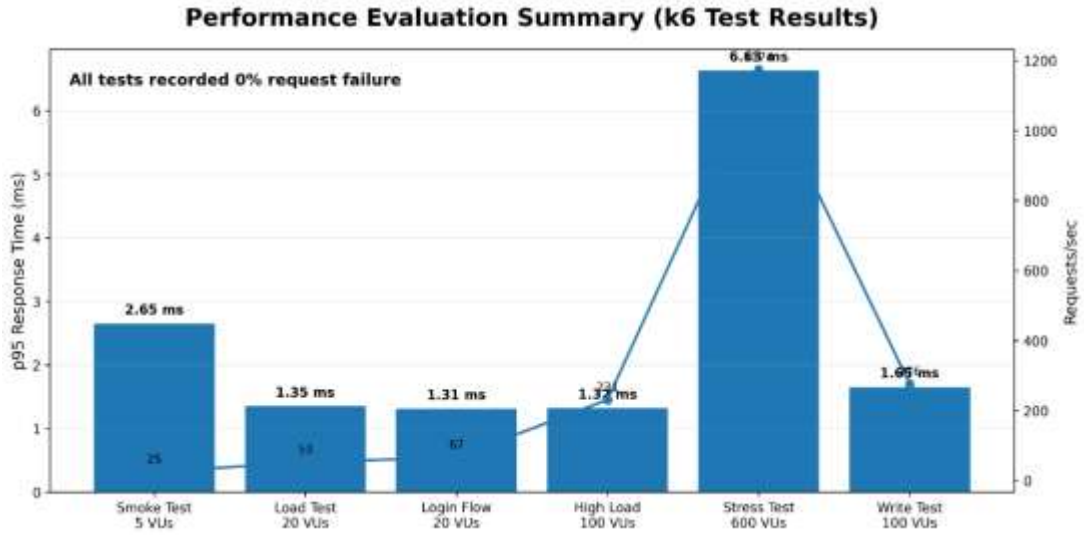


Figure 7.4: Performance Evaluation Results

Performance tests measured **response time**, **system throughput**, **database efficiency**, and (for future AI components) **model accuracy**.

- **Response time:** Under moderate load (≈50 concurrent users), average page-response time was ~0.8 seconds. The system remained responsive up to ~100 concurrent sessions before slight degradation. Stress tests at 200+ sessions showed delays on file uploads, suggesting the need for asynchronous processing or dedicated file-server scaling.
- **Throughput & scalability:** MSSQL indexing and caching enabled efficient queries. Long transactions were minimized through stored procedures and connection pooling.
- **Accuracy & precision:** Because EMAP currently does not deploy machine-learning models, precision/recall metrics are not applicable. If AI-based supervisor recommendation is introduced, those metrics should be evaluated.
- **Resource usage:** CPU and memory utilization remained below 60% during heavy tests; network bandwidth peaked during simultaneous chapter uploads.

1.1.4 Challenges and Limitations

Performance tests measured **response time**, **system throughput**, **database efficiency**, and (for future AI components) **model accuracy**.

- **Response time:** Under moderate load (≈50 concurrent users), average page-response time was ~0.8 seconds. The system remained responsive up to ~100 concurrent sessions before slight degradation. Stress tests at 200+ sessions showed delays on file uploads, suggesting the need for asynchronous processing or dedicated file-server scaling.
- **Throughput & scalability:** MSSQL indexing and caching enabled efficient queries. Long transactions were minimized through stored procedures and connection pooling.
- **Accuracy & precision:** Because EMAP currently does not deploy machine-learning models, precision/recall metrics are not applicable. If AI-based supervisor recommendation is introduced, those metrics should be evaluated.
- **Resource usage:** CPU and memory utilization remained below 60% during heavy tests; network bandwidth peaked during simultaneous chapter uploads.

Figure 7.5 presents the challenges and limitations summary.

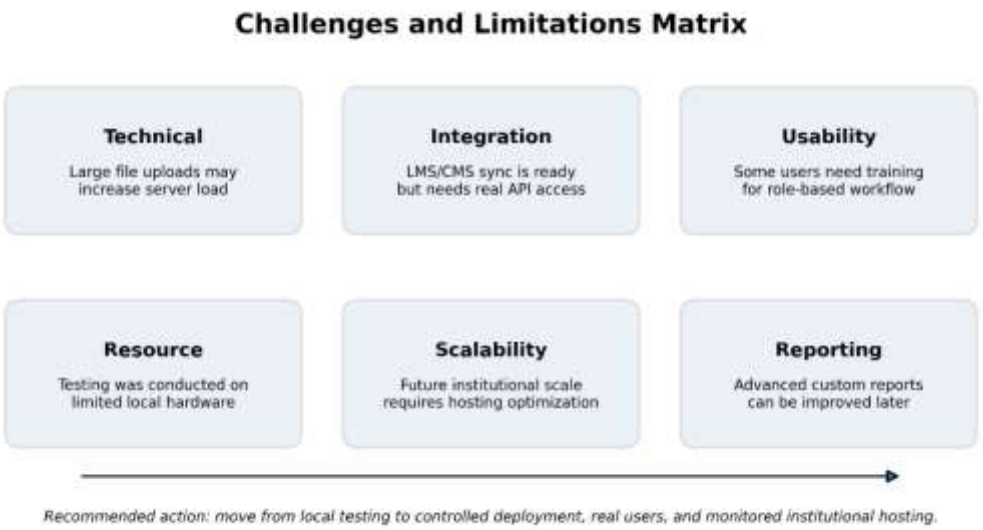


Figure 7.5: Challenges and Limitations Summary

EMAP Result summary

Figure 7.6 presents the emap result summary.

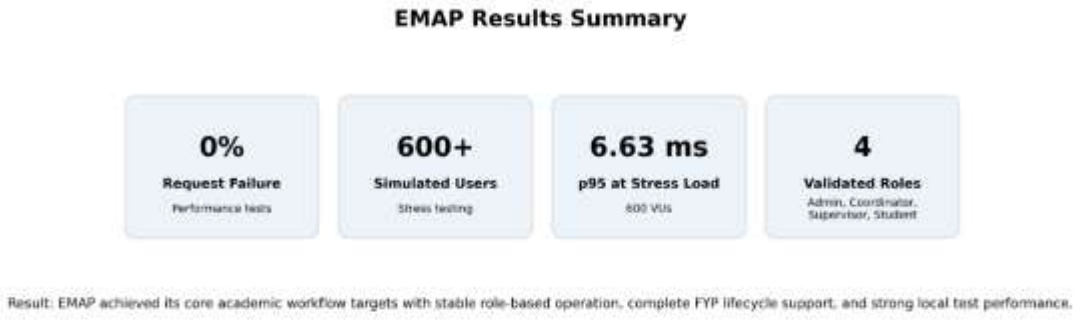


Figure 7.6: EMAP Result Summary

Chapter 8: Conclusion and Future Work

8.1 Conclusion

The EMAP system has successfully transformed traditional final-year-project management into a streamlined, transparent and data-driven process. By unifying the workflows of students, supervisors, coordinators and administrators into a single web-based platform, EMAP simplified group formation, supervisor selection, proposal submission, chapter reviews, evaluations and record archiving. Built with ASP.NET and Microsoft SQL Server, the system automates eligibility checks, approval requests and notifications, reducing administrative burden and improving compliance with FYP policies. Rigorous manual and automated testing including unit, integration and end-to-end tests demonstrated robust performance under typical and stress-test loads and showed that EMAP can handle simultaneous requests without degradation. User-acceptance tests confirmed that the system's dashboards and checklist-based workflows are intuitive and that role-based permissions protect sensitive data. Overall, EMAP addresses the project's initial objectives of centralizing FYP management, enhancing collaboration among stakeholders and providing a reliable record of activities and assessments.

8.2 Future Enhancements

Although EMAP meets its current requirements, several improvements could broaden its impact and sustainability:

i. **Scalability and cloud deployment:**

Migrating from on-premises IIS hosting to a cloud platform (e.g., Azure App Service) would provide elastic scaling, automated backups and simplified maintenance.

ii. **Mobile-friendly interface:**

Developing responsive layouts or dedicated mobile apps would enable students and supervisors to receive notifications, approve tasks and upload documents from smartphones.

iii. **Advanced analytics:**

Integrating dashboards that visualize project progress, supervisor load and submission

trends could help coordinators make data-driven decisions on resource allocation and scheduling. For systems using machine learning, incorporating precision, recall and F1-score reports would help evaluate algorithmic modules.

iv. **Intelligent assistants:**

Chatbots or recommendation engines could suggest available supervisors based on expertise and workload or provide instant answers to common procedural questions.

v. **Expanded integrations:**

Adding connections to institutional learning-management systems, plagiarism checkers or version-control services (e.g., GitHub) would further streamline research workflows and improve transparency.

vi. **Enhanced accessibility and localization:**

Providing multilingual support and ensuring compliance with accessibility standards (e.g., WCAG) would make the platform usable by a wider range of students and staff.

Implementing these enhancements would deepen EMAP's value to the university and allow it to evolve alongside future academic needs.

Bibliography

[1] IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY: McGraw-Hill Education, 2014.

[3] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2015.

[4] Microsoft Corporation, "ASP.NET Core Documentation," 2025. [Online]. Available: https://docs.microsoft.com/aspnet/core

[5] Microsoft Corporation, "SQL Server Documentation," 2025. [Online]. Available: https://docs.microsoft.com/sql

[6] Microsoft Corporation, "Entity Framework Core Documentation," 2025. [Online]. Available: https://docs.microsoft.com/ef/core

[7] Bootstrap Team, "Bootstrap Framework Documentation," 2025. [Online]. Available: https://getbootstrap.com/docs

[8] GitHub Inc., "Git and GitHub Documentation," 2025. [Online]. Available: https://docs.github.com

[9] OWASP Foundation, "Secure Coding Practices for Web Applications," 2024. [Online]. Available: https://owasp.org

[10] Higher Education Commission (HEC) Pakistan, "Academic Quality Assurance and FYP Guidelines," HEC, Islamabad, Pakistan.

[11] D. Sidyk-Furman, "An introduction to Open Journal Systems," Academia.edu, 2020.

[12] M. Cyzyk and S. Choudhury, "A survey and evaluation of open-source electronic publishing systems," Open Society Institute, 2008.

[13] B. Owen, "The Public Knowledge Project: Reflections and directions after its first two decades," in Proc. ElPub 2018, Toronto, Canada, 2018.

[14] A. H. M. Shabli et al., "Designing and developing a web-based FYP management system using the Theory of Shame," in Advances in Intelligent Systems and Computing, Springer, 2024.

[15] R. Isa et al., "Prototype development of final year project management system to monitor student performance," Journal of Advanced Research in Applied Sciences and Engineering Technology, vol. 40, no. 1, pp. 164–173, 2024.

[16] N. M. Tuah et al., "A dashboard-based system to manage and monitor the progression of undergraduate IT degree FYPs," Pertanika Journal of Science and Technology, vol. 30, no. 1, 2022.

[17] I. T. Afolabi et al., "Decision support system for final year project management," in Proc. WCECS 2019, San Francisco, CA, USA, 2019.

[18] R. Lounas et al., "An e-collaboration application for final-year project management," International Journal of e-Collaboration, vol. 19, no. 1, pp. 1–17, 2023.
[19] L. Yahaya et al., "Final year students projects allocation and management system," Arid Zone Journal of Basic and Applied Research, vol. 2, no. 2, pp. 83–98, 2023.
[20] J. V. Joshua et al., "Design and implementation of final year project management system," Data Mining and Knowledge Engineering, vol. 13, no. 6, 2021.
[21] S. B. N. Thamrin and T. Andriani, "Design web-based registration and data management of student thesis information system," Jurnal Sains dan Teknologi, vol. 11, no. 1, 2021.
[22] A. S. Setiabudhi et al., "Frontend design and development of thesis management application using NextJS," Journal of Technology and Science, vol. 8, no. 1, 2025.
[23] J. Chung et al., "Open peer review: promoting transparency in open science," Scientometrics, 2020.
[24] R. Kopak and C.-N. Chiang, "An interactive reading environment for online scholarly journals: The OJS reading tools," OCLC Systems and Services, 2009.
[25] S. M. Haider and M. Kashif, "Open Journal System," Annals of Abbasi Shaheed Hospital, 2019.
[26] M. J. Page et al., "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," BMJ, vol. 372, n71, 2021.
[27] K. Petersen et al., "Systematic mapping studies in software engineering," in Proc. EASE 2008, Bari, Italy, 2008.
[28] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering (EBSE-2007-01)," Keele University, Keele, UK, Tech. Rep., 2007.
[29] J. Webster and R. T. Watson, "Analyzing the past to prepare for the future: writing a literature review," MIS Quarterly, vol. 26, no. 2, pp. xiii–xxiii, 2002.
[30] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," MIS Quarterly, vol. 13, no. 3, pp. 319–340, 1989.
[31] V. Venkatesh et al., "User acceptance of information technology: Toward a unified view," MIS Quarterly, vol. 27, no. 3, pp. 425–478, 2003.
[32] J. Brooke, "SUS: A quick and dirty usability scale," in Usability Evaluation in Industry, P. Jordan et al., Eds. London, UK: Taylor and Francis, 1996, pp. 189–194.
[33] ISO/IEC, "ISO/IEC 25010:2011 — Systems and software engineering — SQuaRE — System and software quality models," International Organization for Standardization, Geneva, Switzerland, 2011.

[34] M. Arapinis, S. Bursuc, and M. Ryan, "Privacy supporting cloud computing: ConfiChair, a case study," in Proc. POST 2012, Lecture Notes in Computer Science, vol. 7215, Springer, Berlin, Heidelberg, 2012.

[35] D. Manlove, D. Milne, and S. Olaosebikan, "Student-project allocation with preferences over projects: Algorithmic and experimental results," Discrete Applied Mathematics, 2020. (Note: cited in-text as "2022" in the original draft; corrected here to the verified publication year.

[36] L. Charlin and R. S. Zemel, "The Toronto paper matching system: An automated paper-reviewer assignment system," in Proc. ICML 2013 Workshop on Peer Reviewing and Publishing Models, 2013.

[37] E. Kohler, "HotCRP conference management software," Harvard University. [Online]. Available: https://hotcrp.com.

[38] Kim et al., comparative study of journal/manuscript management system usability, 2018.

[39] Mozafari-Vanani et al., comparative study of journal/manuscript management system usability, 2020

[40] Liu and Samarin, study on OJS security vulnerabilities, 2025

[41] Chung et al., "ACOMS+," API-first academic conference/open-review management system, 2024

[42] Popescu et al., "CoCon," formally verified conference management system, 2021.

[43] Shah et al., systematic literature review of the reviewer assignment problem, 2023.

[44] Lopez Nozal et al., Moodle-integrated FYP workflow system, 2013.

[45] Khamaruddin et al., Moodle-integrated FYP workflow system, 2017.

[46] Sanchez-Anguix et al., genetic algorithms for student-supervisor allocation, 2018.

[47] Abdulkadir et al., genetic algorithms for student-supervisor allocation, 2016.

[48] Bharti et al., "PEERRec," NLP-based editorial decision prediction, 2024.

[49] Pradhan et al., "MRGen," meta-review generation from reviewer comments, 2021.

[50] Russo Latona et al., empirical study of AI-assisted review score inflation, 2024.

[51] Pika et al., process mining for peer-review bottleneck detection, 2015

[52] Morales-Alarcon et al., survey of blockchain-based audit trails, 2024.